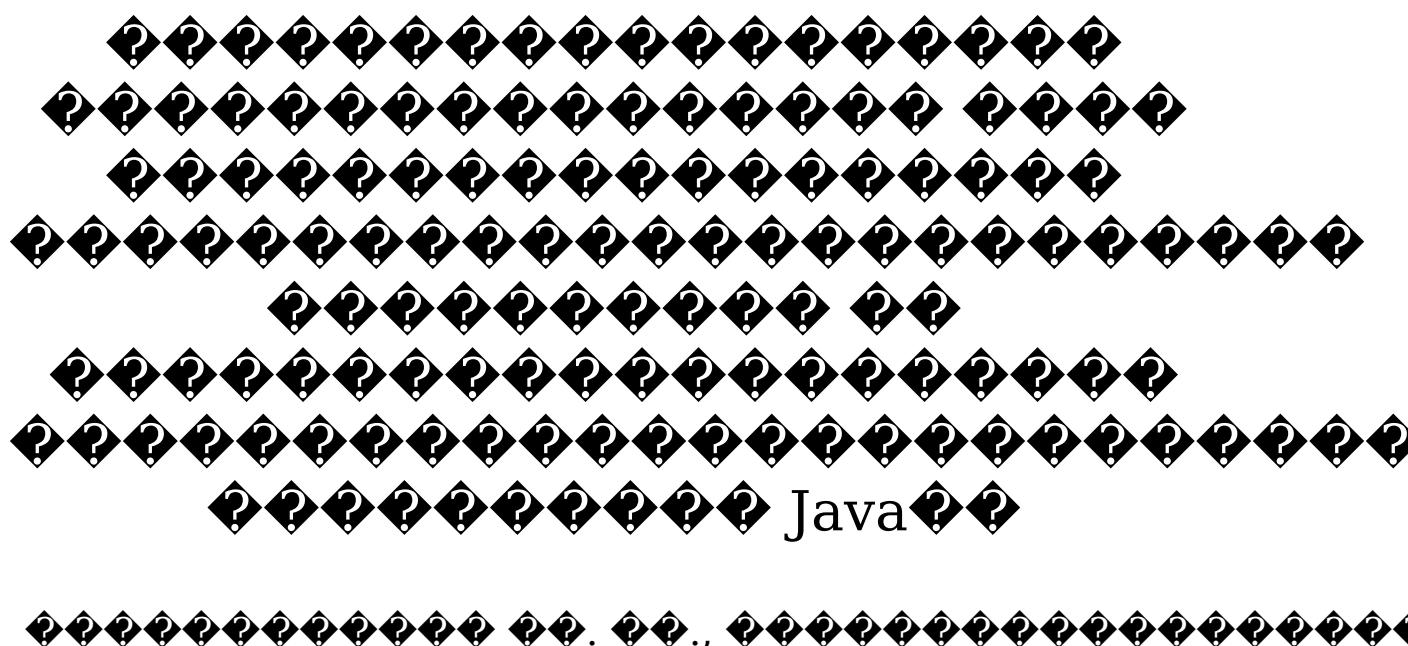




2025

Зміст

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ	4
МЕТОДИЧНІ ВКАЗІВКИ	4
Перелік використаних скорочень	4
Вступ	8
Структура методичних вказівок	8
Вимоги до оформлення звіту	9
Зміст	9
Лабораторні роботи	9
Детальний зміст	10
Лабораторна робота №1	10
Лабораторна робота №2	11
Лабораторна робота №3	11
Лабораторна робота №4	12
Лабораторна робота №5	12
Лабораторна робота №6	12
Лабораторна робота №7	13
Лабораторна робота №8	13
Лабораторна робота №9	13
Лабораторна робота №10	14

Лабораторна робота №1	14
Ознайомлення з компонентами Java SDK. Перша програма:	
компіляція і виконання	14
Тема	14
Мета	14
Теоретичні відомості	15
Хід виконання роботи	16
Індивідуальні завдання	17
Контрольні запитання	18
Перелік посилань	19
Лабораторна робота №2	20
Операції з примітивними типами даних та операторами.	
Операції з масивами	20
Тема	20
Мета	20
Теоретичні відомості	20
Хід виконання роботи	22
Індивідуальні завдання	23
Контрольні запитання	24
Перелік посилань	25
Лабораторна робота №3	25
Об'єктно-орієнтоване програмування на Java	25
Тема	25
Мета	25
Теоретичні відомості	25
Хід виконання роботи	28
Індивідуальні завдання	29
Контрольні запитання	30
Перелік посилань	31
Лабораторна робота №4	31
Операції зі стрічками	31
Тема	31
Мета	32
Теоретичні відомості	32
Хід виконання роботи	34
Індивідуальні завдання	34
Контрольні запитання	36
Перелік посилань	36
Лабораторна робота №5	37
Файли. Потoki введення/виведення	37
Тема	37
Мета	37
Теоретичні відомості	37
Хід виконання роботи	39
Індивідуальні завдання	40

Контрольні запитання	41
Перелік посилань	42
Лабораторна робота №6	43
Застосування класів-колекцій і класів утиліт. Створення автоматизованих тестів	43
Тема	43
Мета	43
Теоретичні відомості	43
Хід виконання роботи	45
Індивідуальні завдання	46
Контрольні запитання	48
Перелік посилань	48
Лабораторна робота №7	49
Створення програм з графічним інтерфейсом користувача	49
Тема	49
Мета	49
Теоретичні відомості	49
Хід виконання роботи	52
Індивідуальні завдання	52
Контрольні запитання	54
Перелік посилань	54
Лабораторна робота №8	55
Використання Java для створення мережевих програм	55
Тема	55
Мета	55
Теоретичні відомості	55
Хід виконання роботи	58
Індивідуальні завдання	58
Контрольні запитання	60
Перелік посилань	61
Лабораторна робота №9	61
Створення програм з кількома підпроцесами	61
Тема	61
Мета	61
Теоретичні відомості	61
Хід виконання роботи	64
Індивідуальні завдання	65
Контрольні запитання	66
Перелік посилань	67
Лабораторна робота №10	67
Робота з базами даних	67
Тема	67
Мета	68
Теоретичні відомості	68

Хід виконання роботи	71
Індивідуальні завдання	72
Контрольні запитання	73
Перелік посилань	74

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Тернопільський національний технічний університет імені Івана Пулюя
Кафедра комп'ютерних систем та мереж

МЕТОДИЧНІ ВКАЗІВКИ

до виконання лабораторних робіт
з дисципліни

«Програмування мовою Java»

для студентів денної та заочної форм навчання
спеціальності F7 «Комп'ютерна інженерія»

Тернопіль – 2025

Укладачі: к.т.н. Луцків А. М., к.т.н. Осухівська Г.М.

Рецензент: Яцишин В. В., к.т.н., завідувач кафедри систем штучного інтелекту та аналізу даних, ТНТУ ім. І. Пулюя

Відповідальний за випуск: зав. каф. комп'ютерних систем та мереж, проф., д.т.н., Лупенко С. А.

Затверджено на засіданні кафедри «Комп'ютерної інженерії»

Протокол № ____ від _____ 2025 р.

Схвалено та рекомендовано до друку методичною комісією факультету комп'ютерно-інформаційних систем і програмної інженерії ТНТУ ім. Івана Пулюя.

Протокол № ____ від _____ 2025 р.

Перелік використаних скорочень

Скорочення	Повна назва	Пояснення
API	Application Programming Interface	Інтерфейс програмування застосунків; набір класів, методів та протоколів для взаємодії між програмними компонентами
CLI	Command Line Interface	Інтерфейс командного рядка; спосіб взаємодії з програмою через текстові команди
CSV	Comma-Separated Values	Формат зберігання табличних даних у текстовому вигляді з роздільником-комою
DB	Database	База даних; організована структура для зберігання, пошуку та обробки даних
FSM	Finite State Machine	Скінченний автомат; обчислювальна модель з кінцевою множиною станів та переходів між ними
GUI	Graphical User Interface	Графічний інтерфейс користувача; спосіб взаємодії з програмою через графічні елементи
HTML	HyperText Markup Language	Мова розмітки гіпертексту; стандартна мова для створення веб-сторінок
HTTP	HyperText Transfer Protocol	Протокол передачі гіпертексту; протокол прикладного рівня для обміну даними у мережі

Скорочення	Повна назва	Пояснення
I/O	Input/Output	Введення/виведення; механізми читання та запису даних
IDE	Integrated Development Environment	Інтегроване середовище розробки; комплексний інструмент для написання, налагодження та тестування коду
JDBC	Java Database Connectivity	Стандартний API мови Java для взаємодії з реляційними базами даних
JDK	Java Development Kit	Набір засобів розробки Java; включає компілятор, JVM, JRE та стандартну бібліотеку
JRE	Java Runtime Environment	Середовище виконання Java; мінімальний набір для запуску Java-застосунків
JSON	JavaScript Object Notation	Текстовий формат обміну даними, що базується на парах «ключ — значення»
JUnit	Java Unit Testing Framework	Фреймворк для написання та виконання модульних тестів у Java
JVM	Java Virtual Machine	Віртуальна машина Java; програмне середовище для виконання байт-коду
LTS	Long-Term Support	Версія програмного забезпечення з тривалою підтримкою безпеки та виправлень

Скорочення	Повна назва	Пояснення
OOP	Object-Oriented Programming	Об'єктно-орієнтоване програмування; парадигма, що базується на поняттях класів та об'єктів
SDK	Software Development Kit	Набір засобів розробки програмного забезпечення для конкретної платформи або технології
SQL	Structured Query Language	Мова структурованих запитів; стандартна мова для роботи з реляційними базами даних
TCP	Transmission Control Protocol	Протокол управління передачею; надійний протокол транспортного рівня з гарантованою доставкою
TCP/IP	Transmission Control Protocol / Internet Protocol	Стек мережевих протоколів, що є основою сучасного Інтернету
UDP	User Datagram Protocol	Протокол датаграм користувача; ненадійний, але швидкий протокол транспортного рівня
URL	Uniform Resource Locator	Уніфікований вказівник ресурсу; стандартний спосіб адресації ресурсів у мережі
UTF	Unicode Transformation Format	Формат перетворення Unicode; стандарт кодування символів (UTF-8, UTF-16)

Вступ

Мова програмування Java займає одне з провідних місць серед технологій розробки програмного забезпечення у світі. Завдяки принципу «напиши один раз — запускай будь-де» (*Write Once, Run Anywhere*), реалізованому через платформу JVM (Java Virtual Machine), Java широко застосовується у корпоративних застосунках, системах Інтернету речей (IoT), мобільних платформах (Android), хмарних сервісах та наукових обчисленнях. Станом на 2025 рік JDK 25 містить широкий спектр сучасних можливостей: записи (*records*), запечатані класи (*sealed classes*), розпізнавання зразків (*pattern matching*), текстові блоки (*text blocks*), віртуальні потоки (*virtual threads*) та структуровану паралельність (*structured concurrency*).

Метою дисципліни «**Програмування мовою Java**» є формування у студентів практичних навичок розробки програмного забезпечення з використанням мови Java, включаючи об'єктно-орієнтоване проєктування, роботу з колекціями та файловою системою, побудову мережевих застосунків, багатопотокове програмування та взаємодію з базами даних через JDBC.

Цей комплект методичних вказівок охоплює десять лабораторних робіт загальним обсягом 36 академічних годин та розрахований на студентів 2-го курсу спеціальності «Комп'ютерна інженерія» (F7). Передбачається, що студенти мають базові знання з алгоритмізації, можуть складати прості алгоритми та мають уявлення про структуру програм (мова C або подібні). Знання мови Java на початку дисципліни не обов'язкове.

Структура методичних вказівок

Методичні вказівки складаються з таких розділів:

1. **Перелік використаних скорочень** — довідковий розділ із поясненням усіх абревіатур.
2. **Вступ** (цей розділ) — мета дисципліни, структура та вимоги до оформлення.
3. **Зміст** — навігаційний перелік усіх розділів.
4. **Лабораторні роботи №1-10** — основна частина, кожна містить: тему, мету, теоретичні відомості, хід виконання, індивідуальні завдання (15 варіантів), контрольні запитання та перелік літератури.

Практичні приклади у методичних вказівках пов'язані з відкритим навчальним репозиторієм `java-iot` (GitHub: <https://github.com/AndriyLutskiv/java-iot>), що містить реальні Java-проєкти для одноплатного комп'ютера CrowPi/Raspberry Pi. Студентам рекомендується переглядати вихідний код цих проєктів як зразок організації коду перед виконанням власних завдань.

Вимоги до оформлення звіту

Звіт з кожної лабораторної роботи оформляється у вигляді текстового документа (формат ODT або PDF) і повинен містити:

1. **Титульний аркуш** — назва університету, кафедра, дисципліна, номер та тема лабораторної роботи, прізвище та ім'я студента, номер групи, рік.
2. **Мета роботи** — одне речення відповідно до методичних вказівок.
3. **Теоретична частина** — стислий конспект (3–5 абзаців) опрацьованого матеріалу; не переписування тексту методичних вказівок, а власне розуміння.
4. **Хід виконання** — опис кожного кроку з поясненням прийнятих рішень.
5. **Вихідний код програми** — повний текст написаних класів у форматі моноширинного шрифту (Courier New, 10 pt).
6. **Результати виконання** — знімки екрану або текстовий вивід програми, що підтверджують коректність роботи.
7. **Відповіді на контрольні запитання** — розгорнуті відповіді (по 3–5 речень) на обрані 5 запитань із переліку.
8. **Висновки** — 3–5 речень щодо набутих навичок та результатів роботи.

Код оформляється відповідно до стандарту Google Java Style Guide: відступ 2 пробіли, відкриваюча дужка на тому ж рядку, коментарі документації (`/** */`) для публічних класів та методів.

Зміст

- [Перелік використаних скорочень](#)
- [Вступ](#)

Лабораторні роботи

№	Тема	Год.	Файл
1	Ознайомлення з компонентами Java SDK. Перша програма: компіляція і виконання	2	lab_01.md

№	Тема	Год.	Файл
2	Операції з примітивними типами даних та операторами.	2	lab_02.md
3	Операції з масивами Об'єктно-орієнтоване програмування на Java	2	lab_03.md
4	Операції зі стрічками	4	lab_04.md
5	Файли. Потоки введення/виведення	4	lab_05.md
6	Застосування класів-колекцій і класів утиліт. Створення автоматизованих тестів	4	lab_06.md
7	Створення програм з графічним інтерфейсом користувача	4	lab_07.md
8	Використання Java для створення мережових програм	6	lab_08.md
9	Створення програм з кількома підпроцесами	4	lab_09.md
10	Робота з базами даних	4	lab_10.md
	Усього	36	

Детальний зміст

Лабораторна робота №1

- Тема
- Мета

- Теоретичні відомості
 - Архітектура платформи Java
 - Модель компіляції та виконання
 - Структура програми Java
 - Компіляція та запуск у командному рядку
 - Стандартна бібліотека та модульна система
 - Типи даних та оператор var
 - Безіменні класи (JDK 25)
- Хід виконання роботи
- Індивідуальні завдання (варіанти 1-15)
- Контрольні запитання (15 запитань)
- Перелік посилань

Лабораторна робота №2

- Тема
- Мета
- Теоретичні відомості
 - Примітивні типи даних
 - Класи-обгортки (Wrapper classes)
 - Оператори
 - Одновимірні масиви
 - Двовимірні масиви
 - Приклад з репозиторію java-iot
- Хід виконання роботи
- Індивідуальні завдання (варіанти 1-15)
- Контрольні запитання (15 запитань)
- Перелік посилань

Лабораторна робота №3

- Тема
- Мета
- Теоретичні відомості
 - Класи та об'єкти
 - Інкапсуляція
 - Наслідування
 - Інтерфейси
 - Поліморфізм
 - Записи (records)
 - Запечатані класи (sealed classes)
 - Приклад з репозиторію java-iot
- Хід виконання роботи
- Індивідуальні завдання (варіанти 1-15)
- Контрольні запитання (15 запитань)
- Перелік посилань

Лабораторна робота №4

- Тема
- Мета
- Теоретичні відомості
 - Клас String
 - Пул рядків (String Pool)
 - Текстові блоки (Text Blocks)
 - StringBuilder та StringBuffer
 - Форматування
 - StringJoiner та String.join
 - Регулярні вирази
 - Приклад з репозиторію java-iot
- Хід виконання роботи
- Індивідуальні завдання (варіанти 1-15)
- Контрольні запитання (15 запитань)
- Перелік посилань

Лабораторна робота №5

- Тема
- Мета
- Теоретичні відомості
 - Ієрархія потоків у java.io
 - java.nio.file — сучасний API
 - BufferedReader з try-with-resources
 - Файлова система: Path та Files
 - Серіалізація об'єктів
 - Формат CSV
 - Приклад з репозиторію java-iot
- Хід виконання роботи
- Індивідуальні завдання (варіанти 1-15)
- Контрольні запитання (15 запитань)
- Перелік посилань

Лабораторна робота №6

- Тема
- Мета
- Теоретичні відомості
 - Java Collections Framework
 - Клас Collections
 - Stream API
 - JUnit 5 — структура тестів
 - Приклад з репозиторію java-iot
- Хід виконання роботи
- Індивідуальні завдання (варіанти 1-15)
- Контрольні запитання (15 запитань)
- Перелік посилань

Лабораторна робота №7

- Тема
- Мета
- Теоретичні відомості
 - Огляд GUI-бібліотек Java
 - Основні компоненти Swing
 - Структура Swing-застосунку
 - Менеджери розміщення
 - Подієва модель
 - Look and Feel
 - Приклад з репозиторію java-iot
- Хід виконання роботи
- Індивідуальні завдання (варіанти 1–15)
- Контрольні запитання (15 запитань)
- Перелік посилань

Лабораторна робота №8

- Тема
- Мета
- Теоретичні відомості
 - Основи мережевої моделі
 - TCP-сервер (ServerSocket)
 - TCP-клієнт (Socket)
 - HTTP-клієнт (java.net.http)
 - Асинхронний HTTP (sendAsync)
 - UDP (датаграми)
 - Приклад з репозиторію java-iot
- Хід виконання роботи
- Індивідуальні завдання (варіанти 1–15)
- Контрольні запитання (15 запитань)
- Перелік посилань

Лабораторна робота №9

- Тема
- Мета
- Теоретичні відомості
 - Потоки виконання (Threads)
 - Стани потоку
 - ExecutorService — пул потоків
 - Callable та Future
 - CompletableFuture
 - Синхронізація: synchronized
 - volatile
 - Атомарні класи
 - Блокування: Lock та ReentrantLock
 - Віртуальні потоки (Java 21+)

- Приклад з репозиторію java-iot
- Хід виконання роботи
- Індивідуальні завдання (варіанти 1-15)
- Контрольні запитання (15 запитань)
- Перелік посилань

Лабораторна робота №10

- Тема
- Мета
- Теоретичні відомості
 - Архітектура JDBC
 - SQLite як навчальна база даних
 - Підключення та базові операції
 - Транзакції
 - Патерн репозиторію (Repository Pattern)
 - Запобігання SQL-ін'єкціям
 - Приклад з репозиторію java-iot
- Хід виконання роботи
- Індивідуальні завдання (варіанти 1-15)
- Контрольні запитання (15 запитань)
- Перелік посилань

Лабораторна робота №1

Ознайомлення з компонентами Java SDK. Перша програма: компіляція і виконання

Тема

Ознайомлення з компонентами Java Development Kit (JDK), засобами компіляції та виконання Java-програм у командному рядку.

Мета

Набути практичних навичок встановлення та налаштування JDK 25, написання, компіляції та запуску першої програми мовою Java шляхом безпосередньої роботи з інструментами командного рядка `javac` та `java`.

Теоретичні відомості

Архітектура платформи Java Платформа Java складається з трьох ключових компонентів:

- **JDK (Java Development Kit)** — повний набір засобів для розробки: включає компілятор `javac`, інструмент документування `javadoc`, налагоджувач `jdb`, засоби профілювання та JRE.
- **JRE (Java Runtime Environment)** — середовище виконання: містить JVM та стандартну бібліотеку класів. Починаючи з JDK 11, окремий дистрибутив JRE більше не поставляється — JDK використовується безпосередньо.
- **JVM (Java Virtual Machine)** — віртуальна машина, що виконує байт-код. Саме JVM забезпечує незалежність від платформи.

Модель компіляції та виконання

Вихідний код (.java)
 ↓ `javac` (компілятор)
Байт-код (.class)
 ↓ `java` (JVM)
Виконання програми

Компілятор `javac` перетворює текст програми (.java) на платформо-незалежний байт-код (.class). Інтерпретатор JVM виконує байт-код, перетворюючи його на машинні інструкції цільової системи (JIT-компіляція).

Структура програми Java

```
// Ім'я файлу: HelloWorld.java
package ua.edu.example; // необов'язковий оголошувач пакету

// Клас – основна структурна одиниця Java
public class HelloWorld {

    // Точка входу: метод main
    public static void main(String[] args) {
        // Виведення рядка на стандартний вихід
        System.out.println("Привіт, Java 25!");
    }
}
```

Ключові правила: - Ім'я публічного класу повинно збігатися з іменем файлу (без розширення). - Метод `main` з сигнатурою `public static void main(String[] args)` є стандартною точкою входу; у JDK 25 допускаються також «безіменні» класи (unnamed classes) для коротких програм. - Пакет (package) — простір імен для організації класів; відповідає структурі директорій.

Компіляція та запуск у командному рядку

```
# Компіляція
javac HelloWorld.java

# Запуск (вказувати .class НЕ потрібно)
java HelloWorld

# З пакетом
javac -d out src/ua/edu/example/HelloWorld.java
java -cp out ua.edu.example.HelloWorld
```

Стандартна бібліотека та модульна система JDK 25 організований за модульною системою (JPMS, введеною у JDK 9). Модуль `java.base` завантажується автоматично і містить пакети `java.lang`, `java.util`, `java.io` та ін. Клас `java.lang.System` надає поля `in`, `out`, `err` для стандартного I/O.

Типи даних та оператор `var` Для локальних змінних з Java 10 підтримується виведення типу через `var`:

```
var message = "Привіт!"; // тип String виводиться автоматично
var count = 42;           // тип int
```

Безіменні класи (JDK 25) У JDK 25 стабілізовано можливість запускати програми без явного оголошення класу та методу `main`:

```
// Файл: Hello.java – запускається командою: java Hello.java
void main() {
    System.out.println("Безіменний клас у JDK 25!");
}
```

Це зручно для навчальних прикладів; у продуктивному коді слід використовувати повну форму.

Хід виконання роботи

1. Перевірити встановлення JDK: виконати команду `java -version` та `javac -version` у терміналі. Переконаватися, що версія відповідає JDK 25.
2. Ознайомитися зі структурою директорії JDK: переглянути вміст `JAVA_HOME/bin`, `JAVA_HOME/lib`. Визначити призначення ключових інструментів (`javac`, `java`, `jar`, `javadoc`).
3. Ознайомитися з навчальним репозиторієм `java-iot`: клонувати або відкрити у браузері файл [core/src/main/java/ua/crowpi/core/launcher/Launcher.java](#). Проаналізувати структуру класу, точку входу `main`, аргументи командного рядка.

4. Переглянути інтерфейс [CrowPiProject.java](#) — звернути увагу на оголошення інтерфейсу, методів та їхні типи повернення.
 5. Налаштувати IDE (IntelliJ IDEA або VSCode з розширенням Extension Pack for Java): створити новий Java-проект, вказати JDK 25 як SDK.
 6. Створити файл відповідно до свого варіанту індивідуального завдання. Дотримуватися структури: оголошення пакету, оголошення класу, метод main.
 7. Скомпілювати програму через IDE або командою `javac <ім'я файлу>.java` у терміналі. Проаналізувати повідомлення про помилки, якщо вони виникнуть.
 8. Запустити програму та перевірити відповідність виводу умові завдання.
 9. Навмисно внести синтаксичну помилку (наприклад, пропустити крапку з комою) та проаналізувати повідомлення компілятора.
 10. Виконати програму з аргументами командного рядка (`java <Клас> arg1 arg2`), якщо передбачено варіантом. Переконаватися, що `args[0]`, `args[1]` обробляються коректно.
 11. Згенерувати документацію Javadoc командою `javadoc -d doc <ім'я файлу>.java`. Переглянути результат у браузері.
 12. Оформити звіт: зафіксувати результати компіляції та запуску (знімки екрану або текстовий вивід).
-

Індивідуальні завдання

Розробити програму мовою Java відповідно до варіанту. Кожна програма: один клас, точка входу main, коректно компілюється та виконується у JDK 25.

Варіант 1. Написати програму, що виводить інформацію про студента (прізвище, ім'я, група, рік вступу) у вигляді форматованої таблиці за допомогою `System.out.printf`. Дані задати константами у методі main.

Варіант 2. Написати програму, що приймає з командного рядка два цілих числа (аргументи `args[0]` та `args[1]`) і виводить їх суму, різницю, добуток та частку. Передбачити перевірку кількості аргументів.

Варіант 3. Написати програму-калькулятор температур: приймає значення у градусах Цельсія з командного рядка (`args[0]`), виводить відповідне значення у Фаренгейтах та Кельвінах з поясненнями.

Варіант 4. Написати програму, що виводить таблицю множення (10×10) у форматі сітки за допомогою вкладених циклів `for` та `System.out.printf` для вирівнювання стовпців.

Варіант 5. Написати програму, що виводить ASCII-арт прямокутника заданих розмірів (рядки та стовпці приймаються з `args[0]` та `args[1]`) із символу *. Межі — *, внутрішня частина — пробіли.

Варіант 6. Написати програму, що обчислює та виводить перші 20

чисел Фібоначчі. Використати цикл `for` та дві змінні для зберігання попередніх значень. Форматувати вивід: кожне число з порядковим номером.

Варіант 7. Написати програму, що приймає рядок з `args[0]` і визначає, чи є він паліндромом (читається однаково в обох напрямках). Ігнорувати регістр символів. Вивести результат з поясненням.

Варіант 8. Написати програму-лічильник: виводить числа від 1 до 100, але замість чисел, кратних 3, виводить «Fizz», кратних 5 — «Buzz», кратних 15 — «FizzBuzz» (класична задача FizzBuzz).

Варіант 9. Написати програму, що обчислює факторіал числа, отриманого з `args[0]`. Передбачити перевірку діапазону (0-20) та повідомлення про помилку при некоректному вводі. Використати тип `long`.

Варіант 10. Написати програму, що виводить просте зображення будиночка символами ASCII: дах (`/\`), стіни (`|`), вікно (`[]`), двері (`||`). Розміри фіксовані, оголошені як константи (`static final int`).

Варіант 11. Написати програму, що приймає три числа з аргументів командного рядка і знаходить найбільше з них без використання методу `Math.max`. Вивести результат з поясненням порівнянь.

Варіант 12. Написати програму, що перевіряє, чи є число, передане у `args[0]`, простим (має ділитися лише на 1 та на себе). Вивести повідомлення та обґрунтування перевірки.

Варіант 13. Написати програму-конвертор одиниць: приймає відстань у кілометрах (`args[0]`) і виводить її у милях, морських милях та метрах. Коефіцієнти оголосити як `static final double`.

Варіант 14. Написати програму, що виводить діагональний хрест `X` розміром `N×N` (`N` приймається з `args[0]`, непарне ціле від 3 до 15). Клітини діагоналей — `*`, решта — пробіли.

Варіант 15. Написати програму, що читає ціле число з `args[0]` і виводить всі його дільники, а також визначає, чи є число «досконалим» (сума дільників, крім самого числа, дорівнює числу). Приклад: $6 = 1 + 2 + 3$.

Контрольні запитання

1. Що таке JDK, JRE та JVM? Чим вони відрізняються та яке відношення мають одне до одного?
2. Що означає принцип «Write Once, Run Anywhere»? За рахунок чого він реалізується у Java?
3. Яку роль відіграє компілятор `javac`? Що є результатом його роботи?

4. Що таке байт-код? Чому він не є машинним кодом конкретного процесора?
 5. Що таке JIT-компілятор і як він підвищує швидкодію Java-програм?
 6. Які обов'язкові елементи має містити будь-яка виконувана Java-програма? Опишіть сигнатуру методу `main`.
 7. Навіщо в Java використовується система пакетів? Яка конвенція назв пакетів прийнята у галузі?
 8. Що відбудеться, якщо ім'я публічного класу не збігається з іменем файлу?
 9. Що таке `classpath` та як його налаштувати при компіляції та запуску програми?
 10. Що таке модульна система JPMS (Project Jigsaw)? У чому її перевага порівняно з `classpath`?
 11. Яке призначення інструменту `javadoc`? Які теги підтримує формат Javadoc-коментарів?
 12. Що таке «безіменні класи» (unnamed classes) у JDK 25? У яких сценаріях вони є доречними?
 13. Чим відрізняється оператор `var` від слабкої типізації? Чи є Java з `var` нестрого типізованою мовою?
 14. Що означає модифікатор `static` у контексті методу `main`?
 15. Яке значення повертає JVM операційній системі при нормальному завершенні програми? Як явно задати код завершення?
-

Перелік посилань

1. Schildt, H. *Java: The Complete Reference* (12th ed.). New York : McGraw-Hill, 2021. 1248 p.
2. Horstmann, C. S. *Core Java, Volume I — Fundamentals* (12th ed.). Hoboken : Pearson, 2022. 928 p.
3. Oracle Corporation. *Java SE 25 Documentation* [Електронний ресурс]. URL: <https://docs.oracle.com/en/java/javase/25/> (дата звернення: 2025-09-01).
4. Oracle Corporation. *Java SE 25 API Specification* [Електронний ресурс]. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/> (дата звернення: 2025-09-01).
5. Oracle Corporation. *The Java™ Tutorials* [Електронний ресурс]. URL: <https://docs.oracle.com/javase/tutorial/> (дата звернення: 2025-09-01).
6. Bloch, J. *Effective Java* (3rd ed.). Boston : Addison-Wesley, 2018. 412 p.
7. Eckel, B. *Thinking in Java* (4th ed.). Upper Saddle River : Prentice Hall, 2006. 1150 p.

Лабораторна робота №2

Операції з примітивними типами даних та операторами. Операції з масивами

Тема

Операції з примітивними типами даних та операторами; операції з одновимірними та двовимірними масивами мови Java.

Мета

Набути практичних навичок використання примітивних типів даних, арифметичних, логічних та побітових операторів, а також одновимірних і двовимірних масивів мови Java шляхом написання та налагодження навчальних програм.

Теоретичні відомості

Примітивні типи даних Java має вісім примітивних типів, що зберігаються безпосередньо у стеку (на відміну від об'єктів):

Тип	Розмір	Діапазон	Приклад літерала
byte	8 біт	-128 ... 127	(byte) 100
short	16 біт	-32 768 ... 32 767	(short) 1000
int	32 біти	$-2^{31} \dots 2^{31}-1$	42, 0xFF
long	64 біти	$-2^{63} \dots 2^{63}-1$	9_000_000_000L
float	32 біти	IEEE 754	3.14f
double	64 біти	IEEE 754	3.14159
char	16 біт	0 ... 65 535 (Unicode)	'A', 'A'
boolean	1 біт	true / false	true

Символ підкреслення в числових літералах (1_000_000) допускається починаючи з Java 7 і покращує читабельність.

Класи-обгортки (Wrapper classes) Кожному примітивному типу відповідає клас-обгортка: Integer, Long, Double, Boolean тощо. Автоупакування (*autoboxing*) і розпакування (*unboxing*) виконуються компілятором автоматично:

```
Integer a = 42;           // autoboxing: int → Integer
int b = a;                // unboxing: Integer → int
```

```
int max = Integer.MAX_VALUE;    // 2 147 483 647
String s = Integer.toBinaryString(255); // "11111111"
```

Оператори Арифметичні: +, -, *, /, % (остача від ділення), ++, --

Порівняння: ==, !=, <, >, <=, >=

Логічні: && (і), || (або), ! (не), ^ (XOR)

Побітові: &, |, ^, ~, <<, >>, >>> (зсув без знаку)

Тернарний: умова ? вираз1 : вираз2

Ділення цілих чисел у Java дає ціле: 7 / 2 == 3. Для отримання дробового результату необхідно перетворення: 7.0 / 2 або (double) 7 / 2.

Одновимірні масиви

// Оголошення та ініціалізація

```
int[] temperatures = new int[10];           // нульові значення за замовчуванням
double[] readings = {36.6, 37.1, 36.8}; // ініціалізатор
```

// Довжина масиву

```
int len = readings.length; // поле, не метод!
```

// Перебір

```
for (int i = 0; i < readings.length; i++) {
    System.out.printf("readings[%d] = %.1f\n", i, readings[i]);
}
```

// Розширений for (foreach)

```
for (double r : readings) {
    System.out.println(r);
}
```

Клас java.util.Arrays містить методи сортування (Arrays.sort), пошуку (Arrays.binarySearch) та порівняння (Arrays.equals).

Двовимірні масиви

// Матриця 3×4

```
int[][] matrix = new int[3][4];
```

// Ініціалізація літералом

```
int[][] identity = {
    {1, 0, 0},
    {0, 1, 0},
    {0, 0, 1}
};
```

// Перебір

```
for (int row = 0; row < matrix.length; row++) {
    for (int col = 0; col < matrix[row].length; col++) {
```

```

        System.out.printf("%4d", matrix[row][col]);
    }
    System.out.println();
}

```

Java підтримує «зубчасті» (jagged) масиви — рядки можуть мати різну довжину.

Приклад з репозиторію java-iot Клас [TemperatureReading.java](#) зберігає показання температури як поля примітивного типу. Клас [Note.java](#) використовує масив частот нот та операції приведення типів для генерації звуку. Рекомендується переглянути ці файли як приклади практичного використання примітивних типів.

Хід виконання роботи

1. Оголосити та ініціалізувати змінні кожного з восьми примітивних типів; вивести їх значення разом із іменем типу (`getClass()` для обгортки або `typeof` через `var`).
 2. Дослідити межі типів: вивести `Integer.MAX_VALUE`, `Integer.MIN_VALUE`, `Long.MAX_VALUE`, `Double.MAX_VALUE`. Перевірити, що відбувається при переповненні (наприклад, `Integer.MAX_VALUE + 1`).
 3. Відкрити у браузері клас `TemperatureReading.java` (посилання вище) та проаналізувати, як у ньому зберігаються показання датчика. Визначити типи всіх полів.
 4. Реалізувати обчислення відповідно до свого варіанту індивідуального завдання, використовуючи відповідні арифметичні оператори.
 5. Проілюструвати роботу побітових операторів: виконати `&`, `|`, `^`, `<<`, `>>` над двома цілими числами, вивести результати у двійковому (`Integer.toBinaryString`) та шістнадцятковому форматі.
 6. Оголосити одновимірний масив відповідно до варіанту, заповнити його значеннями (через цикл або літерал), вивести елементи.
 7. Виконати операції над масивом: знайти мінімум, максимум, суму, середнє. Порівняти ручний пошук з методами `Arrays.sort + readings[0]`.
 8. Реалізувати двовимірний масив відповідно до варіанту (матриця, таблиця тощо).
 9. Перевірити роботу тернарного оператора на 3–5 різних умовах.
 10. Дослідити неявне та явне приведення типів (`int` → `double`, `double` → `int`). Перевірити, чи втрачається точність при зуженні.
 11. Оформити звіт: включити вихідний код та знімки виводу програми.
-

Індивідуальні завдання

Розробити програму мовою Java (один клас, метод main). Обсяг: до 80 рядків коду. Усі дані задати у вигляді масивів або констант; введення з Scanner вітається, але не обов'язкове.

Варіант 1. Задати масив температур (10 значень типу double, від -20 до +40) та обчислити: мінімальну, максимальну, середню температури; кількість днів зі значенням вище середнього. Вивести результати та відсортований масив.

Варіант 2. Задати масив цілих чисел (15 елементів) та знайти: суму елементів на парних позиціях; суму на непарних позиціях; кількість від'ємних елементів; добуток ненульових елементів.

Варіант 3. Задати матрицю 4×4 цілих чисел. Обчислити: суму головної діагоналі; суму побічної діагоналі; максимальний елемент кожного рядка.

Варіант 4. Задати масив показань рівня сигналу (значення типу int, від 0 до 100). Знайти кількість значень у кожному діапазоні: 0-25, 26-50, 51-75, 76-100. Вивести гістограму символами #.

Варіант 5. Задати масив частот нот (7 нот октави, тип double). Обчислити різницю частот між сусідніми нотами, вивести ноту з найбільшою та найменшою частотою. За зразок взяти [Note.java](#).

Варіант 6. Задати масив довжин сторін трикутників (масив з N пар значень double a, b, c). Для кожного трикутника визначити: чи є він дійсним, вид (рівносторонній, рівнобедрений, різносторонній).

Варіант 7. Задати двовимірний масив оцінок (5 студентів × 6 предметів, тип int, 1-100). Обчислити середній бал кожного студента та кожного предмета. Визначити студента з найвищим середнім балом.

Варіант 8. Задати два масиви цілих чисел однакового розміру. Обчислити: скалярний добуток; поелементну суму та різницю (результат — новий масив); кількість позицій, де перший масив більший за другий.

Варіант 9. Задати масив байтових значень (тип byte, 20 елементів). Для кожного значення вивести: десяткове, двійкове та шістнадцяткове представлення. Знайти значення з максимальною кількістю одиничних бітів.

Варіант 10. Задати матрицю 3×3 дійсних чисел. Обчислити транспоновану матрицю та вивести обидві матриці у форматованому вигляді. Перевірити рівність $A[i][j] == A_T[j][i]$.

Варіант 11. Задати масив відліків датчика освітленості (значення типу int, 24 елементи — по одному на кожну годину доби). Знайти: пікове значення; середнє за «денний» час (8:00-20:00); кількість «темних» годин (значення < 10).

Варіант 12. Задати масив рядків-цін у форматі "99.99" (10 елементів, тип String). Перетворити в double (метод Double.parseDouble), знайти суму, мінімум, максимум. Вивести відформатований чек.

Варіант 13. Задати масив цілих чисел та виконати сортування методом бульбашки (не використовувати Arrays.sort). Підрахувати кількість порівнянь та перестановок. Вивести масив до та після сортування.

Варіант 14. Задати квадратну матрицю $N \times N$ (N від 3 до 6). Перевірити, чи є вона симетричною (`matrix[i][j] == matrix[j][i]`). Обчислити суму верхнього та нижнього трикутників окремо.

Варіант 15. Задати масив цілих чисел (20 елементів). Виконати циклічний зсув масиву вправо на K позицій (K від 1 до 5). Вивести масив до та після зсуву. Не використовувати тимчасовий масив великого розміру.

Контрольні запитання

1. Перелічіть усі вісім примітивних типів Java. Яке значення займає кожен у пам'яті?
2. Чим примітивний тип відрізняється від об'єктного (reference) типу? Де вони зберігаються в пам'яті JVM?
3. Що таке автоупакування (autoboxing) та розпакування (unboxing)? Яка небезпека пов'язана із розпакуванням null?
4. Що відбудеться при виконанні `int x = Integer.MAX_VALUE + 1`? Поясніть явище переповнення.
5. Яка різниця між float та double? Коли слід використовувати BigDecimal замість них?
6. Що повертає вираз `7 / 2` і `7.0 / 2`? Поясніть механізм неявного приведення типів.
7. Що означає оператор % для від'ємних чисел? Наведіть приклад: `-7 % 3`.
8. Для чого служить оператор `>>>` (беззнаковий правий зсув) на відміну від `>>`?
9. Що таке «зубчастий» (jagged) масив у Java? Наведіть приклад ситуації, де він є доцільним.
10. Чим `arrays.length` відрізняється від `String.length()`? Чому одне є полем, а інше — методом?
11. Що робить `Arrays.sort` із масивом об'єктів? Який інтерфейс повинні реалізовувати елементи?
12. Як ініціалізуються елементи масиву за замовчуванням залежно від типу?
13. Що таке `ArrayIndexOutOfBoundsException`? Як запобігти цьому винятку?
14. Поясніть різницю між `==` та `equals` при порівнянні масивів.
15. Як за допомогою побітових операцій перевірити, чи встановлено n -й біт цілого числа?

Перелік посилань

1. Horstmann, C. S. *Core Java, Volume I — Fundamentals* (12th ed.). Hoboken : Pearson, 2022. 928 p.
2. Schildt, H. *Java: The Complete Reference* (12th ed.). New York : McGraw-Hill, 2021. 1248 p.
3. Oracle Corporation. *Primitive Data Types — Java SE Tutorial* [Електронний ресурс]. URL: <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/primitives.html> (дата звернення: 2025-09-01).
4. Oracle Corporation. *Arrays — Java SE Tutorial* [Електронний ресурс]. URL: <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/arrays.html> (дата звернення: 2025-09-01).
5. Oracle Corporation. *java.util.Arrays API* [Електронний ресурс]. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/Arrays.html> (дата звернення: 2025-09-01).
6. Bloch, J. *Effective Java* (3rd ed.). Boston : Addison-Wesley, 2018. 412 p.
7. Deitel, P., Deitel, H. *Java: How to Program* (11th ed.). Hoboken : Pearson, 2020. 1040 p.

Лабораторна робота №3

Об'єктно-орієнтоване програмування на Java

Тема

Об'єктно-орієнтоване програмування на Java: класи, об'єкти, інкапсуляція, наслідування, поліморфізм та інтерфейси.

Мета

Набути практичних навичок проектування та реалізації класових ієрархій мовою Java шляхом розробки програм з використанням принципів інкапсуляції, наслідування та поліморфізму.

Теоретичні відомості

Класи та об'єкти Клас у Java — це шаблон, що описує стан (поля) та поведінку (методи) об'єктів. Об'єкт — екземпляр класу, що займає ділянку в купі (*heap*).

```

public class Sensor {
    // Поля (стан)
    private final String name;
    private double lastValue;

    // Конструктор
    public Sensor(String name) {
        this.name = name;
    }

    // Гетер
    public double getLastValue() { return lastValue; }

    // Сетер з перевіркою
    public void setLastValue(double value) {
        if (Double.isNaN(value)) throw new IllegalArgumentException("Значення");
        this.lastValue = value;
    }

    // Перевизначення методу toString
    @Override
    public String toString() {
        return "Sensor{name='" + name + "', value=" + lastValue + '}';
    }
}

```

Інкапсуляція Поля оголошуються private; доступ — через публічні методи-гетери/сетери. Це дозволяє контролювати стан об'єкту та змінювати внутрішню реалізацію без порушення контракту класу.

Наслідування

```

// Базовий клас (суперклас)
public abstract class AbstractDevice {
    protected final String id;

    public AbstractDevice(String id) { this.id = id; }

    // Абстрактний метод – підкласи зобов'язані реалізувати
    public abstract String getStatus();

    // Конкретний метод зі спільною логікою
    public String getId() { return id; }
}

// Підклас
public class TemperatureSensor extends AbstractDevice {
    private double celsius;
}

```

```

    public TemperatureSensor(String id) { super(id); }

    @Override
    public String getStatus() {
        return String.format("T=%.1f°C", celsius);
    }
}

```

Ключові слова: `extends` (наслідування класу), `super(...)` (виклик конструктора суперкласу), `@Override` (анотація, підтверджує перевизначення).

Інтерфейси Інтерфейс визначає контракт поведінки. Один клас може реалізовувати кілька інтерфейсів.

```

public interface Readable {
    double read(); // публічний та абстрактний за замовчуванням
}

```

```

public interface Loggable {
    void log(String message);

    // Дефолтний метод (дозволений з Java 8)
    default void logInfo(String msg) { log("[INFO] " + msg); }
}

```

```

public class PressureSensor extends AbstractDevice
    implements Readable, Loggable {

    @Override public double read() { /* ... */ return 0; }
    @Override public void log(String message) { System.out.println(message); }
    @Override public String getStatus() { return "ok"; }

    public PressureSensor(String id) { super(id); }
}

```

Поліморфізм Посилання на базовий тип або інтерфейс може вказувати на об'єкт будь-якого сумісного підкласу. Виклик методу відбувається через **динамічне зв'язування** (late binding):

```

AbstractDevice[] devices = {
    new TemperatureSensor("T-01"),
    new PressureSensor("P-01")
};

for (AbstractDevice d : devices) {
    // Викликається реалізація відповідного підкласу
    System.out.println(d.getStatus());
}

```

Записи (records, Java 16+) Для незмінних об'єктів-даних зручно використовувати record:

```
public record Reading(String sensorId, double value, long timestamp) {}
```

```
// Компілятор автоматично генерує: конструктор, гетери, equals, hashCode, toString  
Reading r = new Reading("T-01", 36.6, System.currentTimeMillis());  
System.out.println(r.value()); // гетер без "get" префікса
```

Запечатані класи (sealed classes, Java 17+)

```
public sealed interface Shape permits Circle, Rectangle {}  
public record Circle(double radius) implements Shape {}  
public record Rectangle(double w, double h) implements Shape {}
```

Приклад з репозиторію java-iot Інтерфейс [CrowPiProject.java](#) демонструє контракт для всіх IoT-проектів. Клас [ThermometerProject.java](#) реалізує цей інтерфейс. Клас [AlarmFsm.java](#) демонструє поліморфізм через скінченний автомат. Рекомендується вивчити ці файли як зразок побудови класових ієрархій.

Хід виконання роботи

1. Відкрити та проаналізувати інтерфейс [CrowPiProject.java](#) (посилання вище). Визначити: скільки методів оголошено, які параметри та типи повернення, чи є дефолтні методи.
2. Переглянути клас [ThermometerProject.java](#) — виявити, які методи інтерфейсу реалізовано, як використовуються поля та конструктор.
3. Спроекувати власну класову ієрархію відповідно до варіанту: визначити базовий клас (або інтерфейс), 2–3 підкласи, поля та методи.
4. Реалізувати базовий клас або інтерфейс: оголосити поля з private/protected, написати конструктор, гетери, toString, equals, hashCode.
5. Реалізувати підкласи з використанням extends/implements. Перевизначити абстрактні методи з анотацією @Override.
6. Написати метод main, що створює масив об'єктів різних підкласів через посилання на базовий тип та демонструє поліморфну поведінку.
7. Реалізувати принаймні один record для представлення незмінного стану (результат вимірювання, подія тощо).
8. Продемонструвати динамічне зв'язування: викликати перевизначений метод через посилання на суперклас.
9. Перевірити роботу instanceof та розпізнавання зразків (pattern matching): if (device instanceof TemperatureSensor ts).

10. Написати принаймні 3 модульні тести (JUnit 5) для ключових методів розроблених класів.
 11. Оформити звіт: включити UML-діаграму класів (навіть у текстовому вигляді), вихідний код та результати тестування.
-

Індивідуальні завдання

Розробити класову ієрархію з 1 базовим класом або інтерфейсом і 2-3 підкласами. Обсяг: до 120 рядків коду (без тестів). Продемонструвати поліморфізм у методі main.

Варіант 1. Ієрархія «Транспортний засіб»: абстрактний клас `Vehicle` (поля: марка, рік, макс. швидкість); підкласи `Car` (тип кузова), `Truck` (вантажопідйомність), `Motorcycle` (об'єм двигуна). Метод `describe()` у кожному підкласі. Демонстрація у масиві.

Варіант 2. Ієрархія «Геометричні фігури»: інтерфейс `Shape` з методами `area()` та `perimeter()`; record-реалізації `Circle(double r)`, `Rectangle(double w, double h)`, `Triangle(double a, double b, double c)`. Вивести площі та периметри у відсортованому порядку.

Варіант 3. Ієрархія «Банківський рахунок»: абстрактний клас `BankAccount` (номер, баланс); підкласи `SavingsAccount` (відсоткова ставка, нарахування відсотків), `CheckingAccount` (ліміт овердрафту), `DepositAccount` (строк депозиту). Метод `getYearlyIncome()`.

Варіант 4. Ієрархія «Співробітник»: абстрактний клас `Employee` (ім'я, id); підкласи `FullTimeEmployee` (ставка), `PartTimeEmployee` (годинний тариф + кількість годин), `Contractor` (вартість контракту). Абстрактний метод `calculateSalary()`.

Варіант 5. Ієрархія «IoT-датчик» (на основі `java-iot`): інтерфейс `Sensor` з методами `read()` та `getUnit()`; record-реалізації `TemperatureSensor`, `HumiditySensor`, `PressureSensor`. Порівняти з інтерфейсом `CrowPiProject`.

Варіант 6. Ієрархія «Музичний інструмент»: абстрактний клас `Instrument` (назва, матеріал); підкласи `StringInstrument` (кількість струн), `WindInstrument` (тип мундштука), `PercussionInstrument` (тип корпусу). Метод `play()` з різним виводом.

Варіант 7. Ієрархія «Тварина»: абстрактний клас `Animal` (назва, вік); підкласи `Dog` (порода, дресирування), `Cat` (масть, самостійність), `Bird` (розмах крил). Метод `sound()`. Демонстрація поліморфізму через `List<Animal>`.

Варіант 8. Ієрархія «Електронний пристрій»: запечатаний інтерфейс `Device` (sealed) з дозволеними `Laptop`, `Smartphone`, `Tablet`. Кожен — record з полями (бренд, рік, ціна). Метод `describe()` через switch з розпізнаванням зразків.

Варіант 9. Ієрархія «Університет»: абстрактний клас `Person` (ім'я, email); підкласи `Student` (залікова книжка, середній бал), `Lecturer` (кафедра, вчений ступінь), `Administrator` (відділ). Метод `getRole()`.

Варіант 10. Ієрархія «Їжа»: інтерфейс `Edible` (методи `getCalories()`, `isVegetarian()`); record-реалізації `Fruit` (String name, double calories), `Meat` (String type, double calories), `Vegetable` (String name, double calories). Сортування за калорійністю.

Варіант 11. Ієрархія «Фінансовий інструмент»: абстрактний клас `FinancialAsset` (назва, ціна); підкласи `Stock` (тікер, дивіденди), `Bond` (купон, термін), `Cryptocurrency` (символ, капіталізація). Метод `getExpectedReturn()`.

Варіант 12. Ієрархія «Повідомлення»: інтерфейс `Message` (методи `getSender()`, `getContent()`, `getTimestamp()`); record-реалізації `TextMessage`, `EmailMessage`, `PushNotification`. Вивести повідомлення у хронологічному порядку.

Варіант 13. Ієрархія «Ігровий персонаж»: абстрактний клас `Character` (ім'я, здоров'я, рівень); підкласи `Warrior` (атака ближнього бою), `Mage` (запас мани), `Archer` (точність). Абстрактний метод `attack()`.

Варіант 14. Ієрархія «Логістика»: абстрактний клас `Package` (трекінг-номер, вага, адреса); підкласи `StandardPackage`, `ExpressPackage` (термін доставки у годинах), `FragilePackage` (страхова вартість). Метод `calculateShippingCost()`.

Варіант 15. Ієрархія «Медіаконтент»: запечатаний інтерфейс `Media` з реалізаціями `Video` (String title, int durationSeconds), `Audio` (String title, int durationSeconds), `Podcast` (String title, int episodeCount). Метод `describe()` через розпізнавання зразків у `switch`.

Контрольні запитання

1. Що таке клас та об'єкт у Java? Яка різниця між ними з точки зору пам'яті?
2. Що означає інкапсуляція? Навіщо поля оголошуються `private`?
3. Яка різниця між `abstract class` та `interface` в Java? Коли слід обирати одне, а коли інше?
4. Що таке поліморфізм? Поясніть, що означає «динамічне зв'язування» (late binding).
5. Що робить ключове слово `super`? Наведіть два різних способи його використання.
6. Яке призначення анотації `@Override`? Що відбудеться, якщо перевизначити метод із помилкою в підписі без цієї анотації?
7. Що таке `record` у Java? Які методи генеруються компілятором автоматично?
8. Що таке `sealed` клас? Яку проблему він вирішує?

9. Що таке розпізнавання зразків (pattern matching) для instanceof? Як воно спрощує код?
 10. Яка різниця між перевантаженням (overloading) та перевизначенням (overriding) методу?
 11. Що таке конструктор? Чи може клас мати кілька конструкторів? Що таке конструктор за замовчуванням?
 12. Що означає ключове слово final для класу, методу та поля?
 13. Навіщо перевизначати методи equals та hashCode? Яке правило зв'язку між ними?
 14. Що таке дефолтний метод (default method) в інтерфейсі? Коли він є доречним?
 15. Поясніть концепцію «програмування на інтерфейс, а не на реалізацію». Яку перевагу це дає?
-

Перелік посилань

1. Bloch, J. *Effective Java* (3rd ed.). Boston : Addison-Wesley, 2018. 412 p.
2. Horstmann, C. S. *Core Java, Volume I — Fundamentals* (12th ed.). Hoboken : Pearson, 2022. 928 p.
3. Gamma, E., Helm, R., Johnson, R., Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston : Addison-Wesley, 1994. 395 p.
4. Oracle Corporation. *Classes and Objects — Java Tutorials* [Електронний ресурс]. URL: <https://docs.oracle.com/javase/tutorial/java/javaOO/> (дата звернення: 2025-09-01).
5. Oracle Corporation. *Sealed Classes — JEP 409* [Електронний ресурс]. URL: <https://openjdk.org/jeps/409> (дата звернення: 2025-09-01).
6. Oracle Corporation. *Record Classes — JEP 395* [Електронний ресурс]. URL: <https://openjdk.org/jeps/395> (дата звернення: 2025-09-01).
7. Schildt, H. *Java: The Complete Reference* (12th ed.). New York : McGraw-Hill, 2021. 1248 p.

Лабораторна робота №4

Операції зі стрічками

Тема

Операції зі стрічками (рядками): клас String, клас StringBuilder, регулярні вирази та форматування тексту в Java.

Мета

Набути практичних навичок опрацювання рядкових даних засобами стандартної бібліотеки Java шляхом реалізації програм, що використовують методи класу `String`, `StringBuilder`, `StringJoiner`, текстові блоки та регулярні вирази.

Теоретичні відомості

Клас `String` Рядок у Java є об'єктом класу `java.lang.String`. Рядки є **незмінними** (*immutable*): будь-яка операція, що «змінює» рядок, насправді створює новий об'єкт.

```
String s = "Привіт, світе!";
```

```
// Основні методи
```

```
int len    = s.length();           // 14
char c     = s.charAt(0);          // 'П'
String sub = s.substring(8, 13);   // "світ"
String up  = s.toUpperCase();       // "ПРИВІТ, СВІТЕ!"
String tr  = " Java ".strip();      // "Java" (аналог trim(), Unicode-безпечний)
boolean sw = s.startsWith("Прив"); // true
int idx    = s.indexOf("світ");     // 8
String rep = s.replace("світе", "Java"); // "Привіт, Java!"
String[] parts = "a,b,c".split(","); // ["a", "b", "c"]
boolean eq  = "hello".equalsIgnoreCase("HELL0"); // true
```

Важливо: для порівняння рядків завжди використовувати `equals()` або `equalsIgnoreCase()`, а не `==` (оператор `==` порівнює посилання, а не вміст).

Пул рядків (`String Pool`) Рядкові літерали зберігаються у пулі рядків (`String Pool`) у купі. Два однакові літерали вказують на один об'єкт. Метод `intern()` дозволяє помістити рядок у пул вручну.

Текстові блоки (`Text Blocks`, Java 15+)

```
String json = """
    {
        "sensor": "DHT11",
        "temperature": 23.5
    }
    """;
```

Відступи обрізаються автоматично. Текстові блоки зручні для JSON, SQL, HTML у коді.

StringBuilder та StringBuffer `StringBuilder` — змінний (mutable) аналог `String`; ефективний для накопичення рядків у циклі:

```
StringBuilder sb = new StringBuilder();
for (int i = 1; i <= 5; i++) {
    sb.append("Рядок ").append(i).append('\n');
}
String result = sb.toString();
```

`StringBuffer` — потокобезпечний (thread-safe) варіант `StringBuilder`; повільніший через синхронізацію.

Форматування: `String.format` та `formatted`

```
String msg = String.format("Температура: %.1f°C, Вологість: %d%%", 23.5, 65);
// Або через instance-метод (Java 15+):
String msg2 = "Значення: %d".formatted(42);
```

`StringJoiner` та `String.join`

```
// Об'єднання з роздільником
String joined = String.join(", ", "Java", "Python", "C++"); // "Java, Python, C++"

// З префіксом та суфіксом
StringJoiner sj = new StringJoiner(", ", "[", "]");
sj.add("Alpha").add("Beta").add("Gamma");
System.out.println(sj); // [Alpha, Beta, Gamma]
```

Регулярні вирази

```
import java.util.regex.*;

String text = "Код: A-123, Серійний: B-456";

// Перевірка відповідності
boolean match = "abc123".matches("[a-z]+\\d+"); // true

// Пошук усіх збігів
Pattern p = Pattern.compile("[A-Z]-\\d+");
Matcher m = p.matcher(text);
while (m.find()) {
    System.out.println(m.group()); // A-123, потім B-456
}

// Заміна
String clean = text.replaceAll("\\d+", "***"); // маскування чисел
```

Приклад з репозиторію `java-iot` Клас `IrCodeDecoder.java` виконує розбір бітових послідовностей ІЧ-сигналу у рядкові коди команд.

Клас `MelodyLibrary.java` зберігає мелодії у текстовому форматі та розбирає їх у ноти. Рекомендується проаналізувати, як у цих класах використовуються операції зі стрічками.

Хід виконання роботи

1. Ознайомитися з класами `IrCodeDecoder.java` та `MelodyLibrary.java` (посилання вище). Визначити, які методи `String/StringBuilder` у них застосовано.
 2. Написати метод, що демонструє роботу щонайменше 10 методів класу `String`. Виводити вхідний рядок, назву методу та результат.
 3. Порівняти продуктивність конкатенації через `+` та через `StringBuilder` у циклі 100 000 ітерацій. Використати `System.nanoTime()` для вимірювання. Зробити висновки.
 4. Реалізувати завдання відповідно до свого варіанту.
 5. Реалізувати валідацію вхідних даних за допомогою регулярних виразів (наприклад, email, номер телефону, IP-адреса або дата — залежно від варіанту).
 6. Продемонструвати текстові блоки: оголосити рядок у форматі JSON або SQL без екранування, вивести на екран.
 7. Реалізувати метод форматування з `String.format` та `StringJoiner`.
 8. Реалізувати власний метод `reverse(String s)` без використання `StringBuilder.reverse()`. Порівняти результати з вбудованим методом.
 9. Написати 3 JUnit-тести для реалізованих методів рядкової обробки.
 10. Оформити звіт: вихідний код, порівняльна таблиця продуктивності, результати тестування.
-

Індивідуальні завдання

Розробити програму (2–3 класи або один складний клас), що виконує обробку рядкових даних. Обсяг: до 150 рядків коду.

Варіант 1. Реалізувати клас `TextStatistics`, що приймає текст і обчислює: кількість слів, речень, символів (без пробілів), кількість кожної голосної літери. Слова розбивати методом `split` за пробілами та розділовими знаками.

Варіант 2. Реалізувати клас `PasswordValidator` з методами: `isStrong(String pwd)` — пароль ≥ 8 символів, має великі, малі, цифри та спецсимвол; `generateMask(String pwd)` — маскує всі символи крім першого та останнього. Використати регулярні вирази.

Варіант 3. Реалізувати клас `CsvParser`, що розбирає рядок у форматі CSV (з урахуванням полів у лапках, що можуть містити кому) та повертає `List<String>`. Написати зворотній метод `toCsvLine(List<String>)`.

Варіант 4. Реалізувати клас `TemplateFiller`, що приймає шаблон рядка з плейсхолдерами у форматі `{{key}}` та `Map<String, String>` значень, і повертає заповнений рядок. Використати `Pattern` та `Matcher` для пошуку плейсхолдерів.

Варіант 5. Реалізувати клас `MorseCodeConverter` з методами: `toMorse(String text)` — перетворює латинський текст у код Морзе; `fromMorse(String morse)` — зворотнє перетворення. Зберігати таблицю у `Map<Character, String>`.

Варіант 6. Реалізувати клас `LogParser`, що розбирає рядки лог-файлу формату `[РІВЕНЬ] ЧАС – Повідомлення`. Методи: `getLevel()`, `getTime()`, `getMessage()`. Фільтрувати за рівнем за допомогою `enum LogLevel`.

Варіант 7. Реалізувати клас `IrMessageDecoder` (натхненний [IrCodeDecoder.java](#)): приймає рядок із нулів і одиниць довжиною 32 символи, розбиває на 4 байти, виводить кожен у десятковому та шістнадцятковому форматі. Перевірити рядок регулярним виразом `[01]{32}`.

Варіант 8. Реалізувати клас `MarkdownStripper`, що видаляє Markdown-розмітку з рядка: `**текст**` → `текст`, `_текст_` → `текст`, `[текст](url)` → `текст`, заголовки `#` → без `#`. Використати `replaceAll` з регулярними виразами.

Варіант 9. Реалізувати клас `VersionComparator`, що порівнює версії програм у форматі `major.minor.patch` (наприклад, "1.10.3" та "1.9.99"). Методи: `compare(String v1, String v2)` → -1, 0 або 1; `isCompatible(String v1, String v2)` — однакові `major`.

Варіант 10. Реалізувати клас `SqlQueryBuilder` із методами `select(String... cols)`, `from(String table)`, `where(String condition)`, `build()`. Клас накопичує частини запиту в `StringBuilder` та повертає готовий SQL-рядок. Забороняється конкатенація через `+` в циклі.

Варіант 11. Реалізувати клас `PhoneNumberFormatter`, що нормалізує рядки телефонних номерів різних форматів (050) 123-45-67, +380501234567, 050-123-45-67 до єдиного формату +38 (XXX) XXX-XX-XX. Використати регулярні вирази та `replaceAll`.

Варіант 12. Реалізувати клас `RomanNumeralConverter` з методами: `toRoman(int n)` — перетворює ціле (1-3999) у рядок римських цифр; `toArabic(String roman)` — зворотнє перетворення. Валідувати вхідний рядок регулярним виразом.

Варіант 13. Реалізувати клас `HtmlEscaper` з методами: `escape(String text)` — замінює `<`, `>`, `&`, `"`, `'` на HTML-entities; `unescape(String html)` — зворотнє. Написати тести з рядками, що містять усі спецсимволи.

Варіант 14. Реалізувати клас `NoteParser` (натхненний [MelodyLibrary.java](#)): приймає рядок нот формату "C4:500,D4:250,E4:250" (нота:тривалість), повертає список записів `NoteEntry(String name, int durationMs)`. Валідувати формат регулярним виразом.

Варіант 15. Реалізувати клас `WordFrequencyCounter`, що підраховує частоту кожного слова у тексті (без урахування регістру та розділових знаків). Виводити топ-10 найчастіших слів у відсортованому порядку. Порівняти реалізацію на `HashMap` з реалізацією на потоках (`Stream`).

Контрольні запитання

1. Чому рядки у Java є незмінними (`immutable`)? Які переваги це дає?
 2. Чим відрізняється `==` від `equals()` при порівнянні рядків? Наведіть приклад, де `==` дає неочікуваний результат.
 3. Що таке пул рядків (`String Pool`)? Як метод `intern()` пов'язаний з пулом?
 4. Чому конкатенація рядків через `+` у циклі є неефективною? Яку альтернативу слід використовувати?
 5. Чим відрізняється `StringBuilder` від `StringBuffer`? Коли слід використовувати кожен?
 6. Що таке текстові блоки (`text blocks`)? Як обробляється відступ у текстовому блоці?
 7. Що робить метод `strip()` і чим він відрізняється від `trim()`?
 8. Як у Java реалізована підтримка `Unicode` у рядках? Що таке кодова точка (`code point`) та одиниця коду (`code unit`)?
 9. Що таке регулярний вираз? Поясніть синтаксис: `[a-z]+`, `\\d{3}`, `^[A-Z].*\\.\\$`.
 10. Що робить метод `split(String regex, int limit)` з ненульовим параметром `limit`?
 11. Яка різниця між `String.format` та `formatted()`?
 12. Що таке `StringJoiner`? Яку перевагу він має порівняно з `String.join`?
 13. Як порівнювати рядки без урахування регістру? Чи враховує `equalsIgnoreCase` кирилицю?
 14. Що таке `Matcher.group(int)` та як отримати всі групи захоплення?
 15. Коли краще використовувати `contains()`, а коли `indexOf()` — наведіть відмінності.
-

Перелік посилань

1. Horstmann, C. S. *Core Java, Volume I — Fundamentals* (12th ed.). Hoboken : Pearson, 2022. 928 p.
2. Bloch, J. *Effective Java* (3rd ed.). Boston : Addison-Wesley, 2018. 412 p.
3. Oracle Corporation. *java.lang.String API* [Електронний ресурс]. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/String.html> (дата звернення: 2025-09-01).
4. Oracle Corporation. *java.lang.StringBuilder API* [Електронний ресурс]. URL: [https://docs.oracle.com/en/java/javase/25/docs/api/java/la](https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/la)

- (дата звернення: 2025-09-01).
5. Oracle Corporation. *Regular Expressions — Java Tutorials* [Електронний ресурс]. URL: <https://docs.oracle.com/javase/tutorial/essential/regex/> (дата звернення: 2025-09-01).
 6. Oracle Corporation. *Text Blocks — JEP 378* [Електронний ресурс]. URL: <https://openjdk.org/jeps/378> (дата звернення: 2025-09-01).
 7. Schildt, H. *Java: The Complete Reference* (12th ed.). New York : McGraw-Hill, 2021. 1248 p.
 8. Friedl, J. E. F. *Mastering Regular Expressions* (3rd ed.). Sebastopol : O'Reilly Media, 2006. 542 p.

Лабораторна робота №5

Файли. Потoki введення/виведення

Тема

Робота з файловою системою та потоками введення/виведення у Java: пакети `java.io` та `java.nio.file`.

Мета

Набути практичних навичок читання та запису файлів, серіалізації об'єктів та роботи з файловою системою засобами пакетів `java.io` та `java.nio.file` мови Java шляхом реалізації програм обробки файлових даних.

Теоретичні відомості

Ієрархія потоків у `java.io` Потоки поділяються на **байтові** та **символьні**:

Тип	Вхідні	Вихідні
Байтові	<code>InputStream</code>	<code>OutputStream</code>
Символьні	<code>Reader</code>	<code>Writer</code>

Ключові реалізації: - `FileInputStream` / `FileOutputStream` — байтовий файловий I/O - `FileReader` / `FileWriter` — символьний файловий I/O - `BufferedReader` / `BufferedWriter` — буферизовані обгортки для ефективного I/O - `PrintWriter` — зручне форматоване виведення у файл

java.nio.file — сучасний API Починаючи з Java 7, рекомендується використовувати java.nio.file:

```
import java.nio.file.*;
import java.io.IOException;
import java.util.List;

Path path = Path.of("data", "readings.csv"); // Path.of замість new File()

// Запис усіх рядків одразу
List<String> lines = List.of("час,температура", "10:00,23.5", "11:00,24.1");
Files.write(path, lines, StandardCharsets.UTF_8);

// Читання усіх рядків
List<String> read = Files.readAllLines(path, StandardCharsets.UTF_8);

// Читання у рядок
String content = Files.readString(path); // Java 11+

// Запис рядка
Files.writeString(path, "Новий вміст", StandardOpenOption.APPEND);
```

BufferedReader з try-with-resources

```
Path path = Path.of("log.txt");
try (BufferedReader reader = Files.newBufferedReader(path, StandardCharsets.U
    String line;
    while ((line = reader.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException e) {
    System.err.println("Помилка читання: " + e.getMessage());
}
```

Конструкція try-with-resources гарантує закриття ресурсу навіть при виникненні виключення.

Файлова система: Path та Files

```
Path dir = Path.of("output");
Files.createDirectories(dir); // створити директорії рекурсивно
Files.exists(dir);           // перевірка існування
Files.copy(src, dst, StandardCopyOption.REPLACE_EXISTING);
Files.move(src, dst);
Files.delete(path);
long size = Files.size(path); // розмір у байтах

// Перебір файлів у директорії
try (Stream<Path> entries = Files.list(dir)) {
```

```

        entries.filter(p -> p.toString().endsWith(".csv"))
                .forEach(System.out::println);
    }

```

Серіалізація об'єктів

```

import java.io.*;

// Клас для серіалізації повинен реалізовувати Serializable
public record SensorReading(String id, double value) implements Serializable

// Серіалізація (запис у файл)
try (ObjectOutputStream oos = new ObjectOutputStream(
    new FileOutputStream("reading.ser"))) {
    oos.writeObject(new SensorReading("T-01", 36.6));
}

// Десеріалізація (читання з файлу)
try (ObjectInputStream ois = new ObjectInputStream(
    new FileInputStream("reading.ser"))) {
    SensorReading r = (SensorReading) ois.readObject();
    System.out.println(r);
}

```

Формат CSV CSV (*Comma-Separated Values*) — текстовий формат для табличних даних. Кожен рядок — рядок таблиці; поля розділені комою. Перший рядок зазвичай — заголовок.

Приклад з репозиторію java-iot Клас [CsvDataLogger.java](#) записує показання датчиків у CSV-файл за допомогою `PrintWriter` та `BufferedWriter`. Клас [EventFileLogger.java](#) реалізує запис подій у текстовий журнал. Клас [HtmlReportGenerator.java](#) генерує HTML-файл з результатами вимірювань. Рекомендується проаналізувати ці класи перед виконанням завдання.

Хід виконання роботи

1. Відкрити та проаналізувати [CsvDataLogger.java](#) та [EventFileLogger.java](#) (посилання вище). Визначити клас потоку, що використовується, спосіб закриття ресурсів, кодування символів.
2. Створити тестовий файл `test_data.txt` з 10–20 рядками тексту (власне ім'я, дата, числа) засобами `Files.writeString`.
3. Написати метод читання файлу рядок за рядком через `BufferedReader`. Підрахувати кількість рядків, слів та символів.
4. Реалізувати завдання відповідно до свого варіанту.

5. Виконати запис даних у CSV-файл (`data.csv`) та зворотнє читання. Переконавшись, що кодування UTF-8 зберігається (перевірити кирилицю).
 6. Продемонструвати роботу `Files.copy`, `Files.move`, `Files.delete`. Створити директорію `output/`, скопіювати туди файл, перейменувати.
 7. Реалізувати серіалізацію та десеріалізацію об'єктів відповідно до варіанту.
 8. Перехопити та обробити виключення `IOException` та `FileNotFoundException`. Вивести інформативне повідомлення замість стек-трейсу.
 9. Написати 3 JUnit-тести з використанням `@TempDir` (анотація JUnit 5 для тимчасових директорій) для тестування файлових операцій.
 10. Оформити звіт: вихідний код, вміст створених файлів (знімки екрану або вставлений текст), результати тестування.
-

Індивідуальні завдання

Розробити програму (2–3 класи), що реалізує файловий I/O відповідно до варіанту. Обсяг: до 150 рядків коду. Обов'язково: `try-with-resources`, обробка `IOException`, UTF-8 кодування.

Варіант 1. Реалізувати клас `StudentGradeBook`, що зберігає оцінки студентів у CSV-файлі (стовпці: прізвище, ім'я, дисципліна, оцінка). Методи: `addRecord`, `readAll`, `findByStudent(String lastName)`, `averageBySubject(String subject)`.

Варіант 2. Реалізувати клас `TemperatureLogger` (натхненний [Csv-DataLogger.java](#)): записує показання з міткою часу у CSV. Методи: `append(double temp)`, `readLast(int n)`, `getAverage()`. Перевірити роботу з кирилицею у полі коментаря.

Варіант 3. Реалізувати клас `ConfigManager`, що читає та записує конфігурацію з `.properties`-файлу. Методи: `get(String key)`, `set(String key, String value)`, `save()`, `load()`. Зберігати шлях до файлу у полі класу.

Варіант 4. Реалізувати клас `EventLogger` (натхненний [EventFileLogger.java](#)): записує повідомлення у файл журналу з рівнями INFO, WARN, ERROR та мітками часу. Методи: `info`, `warn`, `error`. Записувати у режимі додавання (APPEND).

Варіант 5. Реалізувати клас `HtmlTableGenerator` (натхненний [Html-ReportGenerator.java](#)): приймає список рядків (масиви `String[]`), заголовок та генерує HTML-файл з таблицею. Файл зберегти у кодуванні UTF-8 з метатегом `charset=UTF-8`.

Варіант 6. Реалізувати клас `FileWordCount`, що читає текстовий файл (можна використовувати `Files.lines` зі `Stream`) та підраховує частоту кожного слова. Результат записати у інший файл у форматі слово: кількість, відсортований за зменшенням частоти.

Варіант 7. Реалізувати клас `DirectoryScanner`, що рекурсивно обходить директорію (`Files.walk`), підраховує файли за розширенням та виводить статистику. Результат записати у JSON-подібний текстовий файл.

Варіант 8. Реалізувати клас `SerializableSensorData` із серіалізацією: клас `SensorSnapshot` implements `Serializable` зберігає ідентифікатор, масив показань та мітку часу. Методи: `save(Path p)` та статичний `load(Path p)`. Демонстрація: зберегти → завантажити → порівняти дані.

Варіант 9. Реалізувати клас `MultiFileMerger`, що читає всі `.txt`-файли з вказаної директорії та об'єднує їх у один файл `merged.txt`. Між вмістом кожного файлу вставляти роздільник з ім'ям файлу-джерела.

Варіант 10. Реалізувати клас `CsvToHtmlConverter`: читає CSV-файл з рядком-заголовком та перетворює його у HTML-таблицю з підтримкою кирилиці. Клас `Row` для зберігання рядків CSV.

Варіант 11. Реалізувати клас `LineNumberedFile`: записує будь-який текстовий файл з доданими номерами рядків у форматі `001: текст рядка`. Читає вихідний файл рядок за рядком через `BufferedReader`, записує через `BufferedWriter`.

Варіант 12. Реалізувати клас `BinaryFileAnalyzer`, що читає файл байт за байтом (`FileInputStream`), підраховує частоту кожного значення байта (0-255) та виводить топ-10 найчастіших байтів у шістнадцятковому форматі.

Варіант 13. Реалізувати клас `FileDiff`, що порівнює два текстові файли рядок за рядком та виводить відмінності у форматі уніфікованого `diff` (+ рядок тільки у файлі 2, - рядок тільки у файлі 1).

Варіант 14. Реалізувати клас `DataArchiver` з методами: `compress(Path src, Path dst)` — перетворює текстові рядки у `GZIPOutputStream`; `decompress(Path src, Path dst)` — зворотнє. Порівняти розміри файлів до та після.

Варіант 15. Реалізувати клас `PropertyFileEditor` з підтримкою списків: значення може бути комою-роздільним списком (`key=val1,val2,val3`). Методи: `getList(String key)`, `setList(String key, List<String> values)`. Зберегти у `.properties`-файл.

Контрольні запитання

1. Яка різниця між байтовими (`InputStream/OutputStream`) та символьними (`Reader/Writer`) потоками? Коли слід використовувати кожен тип?
2. Чому при роботі з текстовими файлами важливо явно вказувати кодування (`StandardCharsets.UTF_8`)? Що відбувається, якщо кодування не вказано?
3. Що таке `try-with-resources`? Який інтерфейс повинен реалізовувати ресурс для використання у цій конструкції?

4. Навіщо використовується `BufferedReader/BufferedWriter`? Як буферизація впливає на продуктивність?
 5. Яка різниця між `Path.of` та `new File`? Чому `Path` є кращим вибором у сучасному Java-кодi?
 6. Що таке серіалізація? Яку роль відіграє `serialVersionUID`?
 7. Що таке `transient` поле? Чому деякі поля не варто серіалізувати?
 8. Чим відрізняється `Files.write` від `Files.writeString`? Які `StandardOpenOption` підтримуються?
 9. Що робить `Files.walk` і яку структуру даних вона повертає? Чим відрізняється від `Files.list`?
 10. Як налаштувати `PrintWriter`, щоб він автоматично скидав буфер після кожного `println`?
 11. Що таке `RandomAccessFile`? У яких сценаріях він є необхідним?
 12. Що таке `FileChannel` та `MappedByteBuffer`? Яку перевагу вони дають при роботі з великими файлами?
 13. Як перехопити одразу кілька різних типів виключень в одному блоці `catch`?
 14. Що робить `Files.lines(Path)` і яку перевагу має перед `readAllLines` при роботі з великими файлами?
 15. Як програмно перевірити, чи є шлях файлом, директорією або символьним посиланням?
-

Перелік посилань

1. Horstmann, C. S. *Core Java, Volume II — Advanced Features* (12th ed.). Hoboken : Pearson, 2022. 832 p.
2. Schildt, H. *Java: The Complete Reference* (12th ed.). New York : McGraw-Hill, 2021. 1248 p.
3. Oracle Corporation. *Basic I/O — Java Tutorials* [Електронний ресурс]. URL: <https://docs.oracle.com/javase/tutorial/essential/io/> (дата звернення: 2025-09-01).
4. Oracle Corporation. *java.nio.file.Files API* [Електронний ресурс]. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java.nio/file/Files.html> (дата звернення: 2025-09-01).
5. Oracle Corporation. *File I/O (Featuring NIO.2)* [Електронний ресурс]. URL: <https://docs.oracle.com/javase/tutorial/essential/io/fileio.html> (дата звернення: 2025-09-01).
6. Bloch, J. *Effective Java* (3rd ed.). Boston : Addison-Wesley, 2018. 412 p.
7. Deitel, P., Deitel, H. *Java: How to Program* (11th ed.). Hoboken : Pearson, 2020. 1040 p.

Лабораторна робота №6

Застосування класів-колекцій і класів утиліт. Створення автоматизованих тестів

Тема

Застосування класів-колекцій (List, Set, Map, Queue) та класів-утиліт (Collections, Arrays) Java; написання автоматизованих модульних тестів за допомогою фреймворку JUnit 5.

Мета

Набути практичних навичок використання колекцій Java Collections Framework та написання автоматизованих модульних тестів із застосуванням JUnit 5 шляхом реалізації та тестування програм обробки даних.

Теоретичні відомості

Java Collections Framework Основні інтерфейси та їх реалізації:

Інтерфейс	Реалізації	Особливості
List<E>	ArrayList, LinkedList	Впорядкована колекція, допускає дублікати
Set<E>	HashSet, LinkedHashSet, TreeSet	Множина без дублікатів
Map<K, V>	HashMap, LinkedHashMap, TreeMap	Пари «ключ — значення»
Queue<E>	ArrayDeque, PriorityQueue	Черга FIFO або пріоритетна
Deque<E>	ArrayDeque	Двобічна черга (стек + черга)

```
import java.util.*;
```

```
// List
```

```
List<String> names = new ArrayList<>(List.of("Іван", "Марія", "Олег"));  
names.add("Ганна");  
names.remove("Олег");
```

```

Collections.sort(names);

// Map
Map<String, Integer> scores = new HashMap<>();
scores.put("Іван", 95);
scores.put("Марія", 88);
int score = scores.getOrDefault("Петро", 0); // 0, якщо ключ відсутній

// Set
Set<String> unique = new HashSet<>(names);
unique.add("Іван"); // дублікат – проігнорується

// Незмінні колекції (Java 9+)
List<String> immutable = List.of("A", "B", "C");
Map<String, Integer> immutableMap = Map.of("a", 1, "b", 2);

```

Клас Collections

```

List<Integer> nums = new ArrayList<>(List.of(5, 3, 8, 1, 9));

Collections.sort(nums); // [1, 3, 5, 8, 9]
Collections.reverse(nums); // [9, 8, 5, 3, 1]
Collections.shuffle(nums); // випадковий порядок
int max = Collections.max(nums); // максимум
Collections.frequency(nums, 5); // кількість входжень
List<Integer> unmod = Collections.unmodifiableList(nums); // тільки для читання

```

Stream API (Java 8+)

```

List<String> names = List.of("Анна", "Богдан", "Василь", "Аліна");

// Фільтрація + перетворення + збирання
List<String> filtered = names.stream()
    .filter(n -> n.startsWith("A"))
    .map(String::toUpperCase)
    .sorted()
    .collect(Collectors.toList());
// ["АЛІНА", "АННА"]

// Агрегація
double avg = names.stream()
    .mapToInt(String::length)
    .average()
    .orElse(0.0);

// Групування
Map<Integer, List<String>> byLength = names.stream()
    .collect(Collectors.groupingBy(String::length));

```

JUnit 5 — структура тестів

```
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class CalculatorTest {

    @BeforeEach
    void setUp() {
        // ініціалізація перед кожним тестом
    }

    @Test
    @DisplayName("Сума двох позитивних чисел")
    void addPositiveNumbers() {
        assertEquals(5, Calculator.add(2, 3), "2 + 3 повинно дорівнювати 5");
    }

    @Test
    void divideByZeroThrows() {
        assertThrows(ArithmeticException.class, () -> Calculator.divide(1, 0))
    }

    @ParameterizedTest
    @ValueSource(ints = {1, 3, 5, 7, 9})
    void oddNumbers(int n) {
        assertTrue(n % 2 != 0, n + " повинно бути непарним");
    }
}
```

Ключові анотації JUnit 5: @Test, @BeforeEach, @AfterEach, @BeforeAll, @AfterAll, @DisplayName, @ParameterizedTest, @ValueSource, @CsvSource, @Disabled.

Основні методи Assertions: assertEquals, assertNotEquals, assertTrue, assertFalse, assertNull, assertNotNull, assertThrows, assertAll, assertEquals.

Приклад з репозиторію java-iot Клас [MelodyLibrary.java](#) зберігає мелодії у Map<String, Melody>. Клас [LeaderBoard.java](#) реалізує рейтингову таблицю через TreeMap/List. Тест [PropertiesLoaderTest.java](#) демонструє написання JUnit 5 тестів у стилі репозиторію. Рекомендується проаналізувати всі три класи.

Хід виконання роботи

1. Відкрити та проаналізувати MelodyLibrary.java, LeaderBoard.java та PropertiesLoaderTest.java (посилання вище). Визначити типи

- колекцій та методи, що використовуються.
2. Написати демонстраційний клас, що показує роботу ArrayList, LinkedList, HashSet, TreeSet, HashMap, TreeMap. Для кожної колекції показати: додавання, видалення, пошук, перебір.
 3. Порівняти продуктивність ArrayList та LinkedList при вставці на початок 100 000 елементів. Використати System.nanoTime(). Пояснити результат.
 4. Реалізувати завдання відповідно до свого варіанту.
 5. Реалізувати щонайменше одну операцію з використанням Stream API: фільтрацію, сортування та групування (метод groupingBy).
 6. Написати щонайменше 10 JUnit 5 тестів для реалізованих методів. Використати @ParameterizedTest з @CsvSource для принаймні 2 тестів.
 7. Використати @BeforeEach для ініціалізації тестових даних та assertThrows для перевірки виключень.
 8. Перевірити покриття коду тестами (Coverage) в IDE (IntelliJ: Run with Coverage). Прагнути до $\geq 80\%$ покриття класу з бізнес-логікою.
 9. Оформити звіт: вихідний код, результати JUnit (кількість тестів, кількість успішних), знімок звіту покриття.
-

Індивідуальні завдання

Розробити програму (2–3 класи), що використовує колекції Java, та написати не менше 10 JUnit 5 тестів. Обсяг коду: до 150 рядків (без тестів).

Варіант 1. Реалізувати клас StudentRegistry для реєстру студентів: зберігати у Map<String, Student> (ключ — залікова книжка). Методи: add, remove, findByName(String) → List<Student>, topStudents(int n) — топ-N за середнім балом. Тести: коректність додавання, дублікат, пошук, топ-список.

Варіант 2. Реалізувати клас LibraryCatalog для каталогу книг: зберігати у Map<String, List<Book>> (ключ — автор). Методи: addBook, findByTitle(String), getByAuthor(String), getMostProlificAuthor(). Тести: порожній каталог, однаковий автор, пошук по рядку.

Варіант 3. Реалізувати клас PlaylistManager (натхненний [MelodyLibrary.java](#)): Map<String, List<String>> (жанр → список пісень). Методи: addSong, removeSong, shuffle(String genre), getByGenre. Тести: невідомий жанр, перетасовка.

Варіант 4. Реалізувати клас ScoreBoard (натхненний [LeaderBoard.java](#)): рейтинг гравців у TreeMap<Integer, String> (рахунок → ім'я). Методи: addScore, getTop(int n), getRank(String name). Тести: рівний рахунок двох гравців, негативний рахунок.

Варіант 5. Реалізувати клас FrequencyCounter<T> із generic-параметром: підраховує кількість кожного елемента у Map<T, Integer>. Методи:

`add(T)`, `getCount(T)`, `getMostFrequent()`, `getAll()` — відсортовані за частотою. Тести: рядки, числа.

Варіант 6. Реалізувати клас `UniqueWordDictionary`: зберігає унікальні слова у `TreeSet<String>`. Методи: `addWord`, `remove`, `startsWith(String prefix)` → підмножина, `size`. Тести: нечутливість до регістру, порядок сортування.

Варіант 7. Реалізувати клас `TaskQueue` на основі `ArrayDeque<Task>` (клас `Task` із пріоритетом та описом). Методи: `enqueue`, `dequeue`, `peek`, `processAll`. Альтернативна реалізація з `PriorityQueue`. Тести: порожня черга, порядок видачі.

Варіант 8. Реалізувати клас `EventBus` (паттерн «Спостерігач»): `Map<String, List<Runnable>>` (тема → список обробників). Методи: `subscribe(String topic, Runnable handler)`, `publish(String topic)`, `unsubscribe`. Тести: публікація без підписників, кілька підписників.

Варіант 9. Реалізувати клас `InventoryManager` для складу: `Map<String, Integer>` (товар → кількість). Методи: `addStock`, `removeStock`, `getLowStock(int threshold)`, `getTotalValue(Map<String, Double> prices)`. Тести: від’ємний залишок, порожній склад.

Варіант 10. Реалізувати клас `PhoneBook`: `Map<String, Set<String>>` (ім’я → набір номерів телефонів). Методи: `addContact`, `addPhone`, `removeContact`, `findByPhone(String)` — знайти ім’я за номером. Тести: кілька номерів, видалення.

Варіант 11. Реалізувати клас `SensorDataBuffer<T>` на основі `ArrayDeque` з обмеженням розміру (circular buffer). Методи: `push(T)` (при переповненні — видаляти найстаріший), `toList()`, `average()` (для числових типів через `Number`). Тести: переповнення.

Варіант 12. Реалізувати клас `WordCloud`: підраховує частоту слів у тексті (`Map<String, Long>` через `Collectors.counting`). Методи: `analyze(String text)`, `topN(int n)`, `wordsLongerThan(int len)`. Використати `Stream API` у всіх методах. Тести: порожній рядок, регістр.

Варіант 13. Реалізувати клас `ProductCatalog`: `List<Product>` де `Product record(String name, String category, double price)`. Методи через `Stream API`: `filterByCategory`, `sortByPrice`, `averagePriceByCategory()` → `Map<String, Double>`. Тести: порожній каталог, сортування.

Варіант 14. Реалізувати клас `GraphAdjacency` для орієнтованого графа: `Map<String, Set<String>>` (вузол → множина сусідів). Методи: `addEdge`, `removeEdge`, `getNeighbors`, `hasCycle()` (обхід DFS). Тести: самопетля, граф без циклу, граф з циклом.

Варіант 15. Реалізувати клас `MultiSetCounter<T>`: мультимножина, що дозволяє кілька екземплярів одного значення. Зберігати у `HashMap<T, Integer>`. Методи: `add(T, int count)`, `remove(T, int count)`, `entrySet()` відсортований. Тести: від’ємна кількість, нульова кількість.

Контрольні запитання

1. Яка різниця між `ArrayList` та `LinkedList` з точки зору складності операцій? Коли слід обирати кожен?
2. Яка різниця між `HashSet`, `LinkedHashSet` та `TreeSet`? Який порядок обходу у кожному?
3. Чому `HashMap` не гарантує порядку ітерації? Яка реалізація зберігає порядок вставки?
4. Що таке `equals` та `hashCode` у контексті `HashMap`? Що станеться, якщо два об'єкти рівні за `equals`, але мають різні `hashCode`?
5. Що таке незмінні колекції (`List.of`, `Map.of`)? Яку виключення кидають при спробі модифікації?
6. Яка різниця між `Collections.unmodifiableList` та `List.of`?
7. Що таке `Comparator` та `Comparable`? Як реалізувати нестандартне сортування?
8. Що таке `Stream` у Java? Яка різниця між проміжними (`intermediate`) та термінальними (`terminal`) операціями?
9. Що таке `Collectors.groupingBy`? Наведіть приклад групування списку об'єктів за полем.
10. Які анотації `JUnit 5` відповідають за: ініціалізацію перед кожним тестом, перевірку виключення, параметризований тест?
11. Що таке `assertAll` у `JUnit 5`? Чим він відрізняється від кількох окремих `assertEquals`?
12. Що таке покриття коду (`code coverage`)? Яка різниця між покриттям рядків і покриттям гілок?
13. Що таке `@TempDir` у `JUnit 5`? Для яких тестів він є доречним?
14. Чому не слід тестувати приватні методи безпосередньо? Як тестувати поведінку, що залежить від приватного методу?
15. Що таке `PriorityQueue`? Який природний порядок вона використовує за замовчуванням?

Перелік посилань

1. Bloch, J. *Effective Java* (3rd ed.). Boston : Addison-Wesley, 2018. 412 p.
2. Horstmann, C. S. *Core Java, Volume I — Fundamentals* (12th ed.). Hoboken : Pearson, 2022. 928 p.
3. Oracle Corporation. *Collections — Java Tutorials* [Електронний ресурс]. URL: <https://docs.oracle.com/javase/tutorial/collections/> (дата звернення: 2025-09-01).
4. Oracle Corporation. *java.util.stream — Stream API* [Електронний ресурс]. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/stream/summary.html> (дата звернення: 2025-09-01).
5. JUnit Team. *JUnit 5 User Guide* [Електронний ресурс]. URL: <https://junit.org/junit5/docs/current/user-guide/> (дата звернення: 2025-09-01).
6. Schildt, H. *Java: The Complete Reference* (12th ed.). New York :

- McGraw-Hill, 2021. 1248 p.
7. Warburton, R. *Java 8 Lambdas*. Sebastopol : O'Reilly Media, 2014. 182 p.

Лабораторна робота №7

Створення програм з графічним інтерфейсом користувача

Тема

Створення програм з графічним інтерфейсом користувача (GUI) засобами бібліотеки Swing мови Java.

Мета

Набути практичних навичок розробки застосунків з графічним інтерфейсом користувача з використанням бібліотеки Swing шляхом реалізації віконних компонентів, менеджерів розміщення та обробників подій.

Теоретичні відомості

Огляд GUI-бібліотек Java У Java існує кілька підходів до побудови GUI: - **Swing** (javax.swing) — класична бібліотека, вбудована у JDK. Платформо-незалежна завдяки власній відрисовці компонентів. - **JavaFX** — сучасніший фреймворк із підтримкою CSS, FXML та анімацій. З JDK 11 поставляється окремо (OpenJFX). - **SWT** (Standard Widget Toolkit) — Eclipse-бібліотека з нативними компонентами ОС.

У цій лабораторній роботі використовується **Swing** як найдоступніший варіант без додаткових залежностей.

Основні компоненти Swing

Клас	Призначення
JFrame	Головне вікно застосунку
JPanel	Контейнер для групування компонентів
JLabel	Текстова мітка
JButton	Кнопка
JTextField	Однорядкове текстове поле
JTextArea	Багаторядкове текстове поле
JCheckBox	Прапорець

Клас	Призначення
JRadioButton	Перемикач
JComboBox	Випадний список
JList	Список елементів
JTable	Таблиця
JMenuBar / JMenu / JMenuItem	Рядок меню
JFileChooser	Діалог вибору файлу
JOptionPane	Стандартні діалоги (info, error, input)

Структура Swing-застосунку

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class MainWindow extends JFrame {

    private JTextField inputField;
    private JLabel resultLabel;

    public MainWindow() {
        super("Мій застосунок");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(400, 300);
        setLocationRelativeTo(null); // центр екрану

        // Ініціалізація компонентів
        inputField = new JTextField(20);
        JButton calcButton = new JButton("Обчислити");
        resultLabel = new JLabel("Результат з'явиться тут");

        // Обробник події кліку
        calcButton.addActionListener(e -> onCalculate());

        // Розміщення компонентів
        JPanel panel = new JPanel(new FlowLayout());
        panel.add(new JLabel("Введіть значення:"));
        panel.add(inputField);
        panel.add(calcButton);
        panel.add(resultLabel);

        add(panel);
        setVisible(true);
    }

    private void onCalculate() {
        String text = inputField.getText().strip();
    }
}
```

```

        if (text.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Введіть значення!", "Помилка");
            return;
        }
        resultLabel.setText("Результат: " + text.toUpperCase());
    }

    public static void main(String[] args) {
        // Swing-компоненти слід створювати у EDT (Event Dispatch Thread)
        SwingUtilities.invokeLater(MainWindow::new);
    }
}

```

Менеджери розміщення (Layout Managers)

Менеджер	Опис
FlowLayout	Розміщення зліва направо, перенос на новий рядок
BorderLayout	5 зон: North, South, East, West, Center
GridLayout	Рівна сітка N×M
GridBagLayout	Гнучка сітка з обмеженнями
BoxLayout	Вертикальне або горизонтальне розміщення

```

JPanel panel = new JPanel(new BorderLayout());
panel.add(new JLabel("Заголовок"), BorderLayout.NORTH);
panel.add(new JScrollPane(textArea), BorderLayout.CENTER);
panel.add(okButton, BorderLayout.SOUTH);

```

Подієва модель Swing використовує патерн «Спостерігач» (Observer): компонент генерує подію → слухачі (listeners) обробляють її. З Java 8 зручно використовувати лямбди замість анонімних класів:

```

button.addActionListener(e -> System.out.println("Натиснуто!"));
textField.addKeyListener(new KeyAdapter() {
    @Override
    public void keyPressed(KeyEvent e) {
        if (e.getKeyCode() == KeyEvent.VK_ENTER) onCalculate();
    }
});

```

Look and Feel

```

// Системний L&F (схожий на нативні вікна ОС)
UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());

```

Приклад з репозиторію java-iot Клас `LauncherUI.java` реалізує графічний лаунчер для вибору та запуску IoT-проектів. Клас `ProjectRunner.java` виконує вибраний проект у фоновому потоці. Рекомендується

проаналізувати, як у них організовано компоновку вікна, обробку подій та взаємодію з фоновим потоком.

Хід виконання роботи

1. Відкрити та проаналізувати `LauncherUI.java` та `ProjectRunner.java` (посилання вище). Визначити: які компоненти Swing використано, як організовано EDT, як фоновий потік оновлює UI.
 2. Створити головне вікно (`JFrame`) з заголовком, мінімальним розміром `600x400 px` та розміщенням у центрі екрану.
 3. Реалізувати компоненти форми відповідно до варіанту. Використати принаймні 3 різних типи компонентів та 2 різні менеджери розміщення.
 4. Реалізувати обробники подій для всіх інтерактивних елементів (кнопки, поля введення, список). Для кнопок — через лямбду або `ActionListener`.
 5. Реалізувати валідацію введених даних з виведенням повідомлень через `JOptionPane`.
 6. Додати рядок меню (`JMenuBar`) з розділами «Файл» (Відкрити, Зберегти, Вихід) та «Допомога» (Про програму). Реалізувати пункти «Вихід» та «Про програму».
 7. Реалізувати діалог «Про програму» з назвою застосунку, версією та автором (через `JOptionPane` або `JDialog`).
 8. Переконавшись, що всі компоненти створюються та оновлюються в EDT (`SwingUtilities.invokeLater`). Продемонструвати проблему при виклику UI поза EDT (якщо актуально для варіанту).
 9. Застосувати Look and Feel системного стилю.
 10. Зробити знімки екрану головного вікна, діалогів та результатів роботи програми.
 11. Оформити звіт: вихідний код, знімки екрану з поясненнями, опис подієвої моделі застосунку.
-

Індивідуальні завдання

Розробити Swing-застосунок (2–3 класи). Обсяг: до 150 рядків коду. Обов'язково: щонайменше 3 типи компонентів, рядок меню, валідація, EDT.

Варіант 1. Калькулятор: 4 поля (`JTextField`) для двох операндів, знаку операції (`JComboBox`: `+`, `-`, `*`, `/`) та результату. Кнопки «Обчислити» та «Очистити». Обробка ділення на нуль через `JOptionPane`.

Варіант 2. Записник: `JTextArea` зі `JScrollPane`, кнопки «Відкрити файл», «Зберегти файл» через `JFileChooser`. Відображення шляху до поточного файлу у заголовку вікна. Питання про збереження перед виходом.

Варіант 3. Конвертор одиниць: JComboBox для вибору типу (відстань, температура, маса), два JTextField (вхід/вихід), кнопка «Конвертувати». При зміні значення у першому полі — автоматичний перерахунок через DocumentListener.

Варіант 4. Список завдань (To-Do): JList зі списком завдань, JTextField для введення нового завдання, кнопки «Додати», «Видалити», «Позначити виконаним» (зміна кольору рядка через ListCellRenderer).

Варіант 5. Тест-вікторина: питання відображається у JLabel, 4 варіанти відповіді — JRadioButton у ButtonGroup. Кнопки «Далі» та «Завершити». Після останнього питання — діалог з результатом (кількість правильних відповідей).

Варіант 6. Телефонна книга: JTable зі стовпцями «Ім'я», «Телефон», «Email»; кнопки «Додати», «Видалити», «Знайти». Пошук фільтрує рядки таблиці за введеним текстом через TableRowSorter.

Варіант 7. Реєстраційна форма: поля JTextField для імені, прізвища, email; JComboBox для курсу (1–6); JCheckBox для «Погоджуюсь з умовами»; кнопка «Зареєструватися» (активна лише при позначеному прапорці). Підтвердження реєстрації — JOptionPane.

Варіант 8. Генератор паролів: JSlider для довжини (8–32), JCheckBox для включення: великі, малі, цифри, спецсимволи. Кнопка «Згенерувати» → відображення у JTextField (тільки для читання). Кнопка «Копіювати» → буфер обміну (Toolkit.getDefaultToolkit().getSystemClipboard).

Варіант 9. Прості нотатки: вікно з вкладками (JTabbedPane), кожна вкладка — JTextArea. Кнопки «Додати вкладку», «Закрити поточну». Зберігати вміст вкладок у Map<String, String>.

Варіант 10. Натхненний [LauncherUI.java](#): лаунчер програм. JList зі списком операцій (назви власних навчальних класів). При виборі — опис у JTextArea. Кнопка «Запустити» → виконання у SwingWorker.

Варіант 11. Прогрес-бар задачі: кнопка «Почати завантаження», JProgressBar від 0 до 100%. Симуляція через javax.swing.Timer (збільшення на 5% кожні 200 мс). Після завершення — JOptionPane з повідомленням. Кнопка «Скасувати».

Варіант 12. Форма звіту: JTextField для назви, JTextArea для опису, JSpinner для дати (модель SpinnerDateModel). Кнопка «Попередній перегляд» — виводить форматований звіт у новий JDialog з JTextArea.

Варіант 13. Малювалка: JPanel з перевизначеним paintComponent, JColorChooser для вибору кольору пера, JSlider для товщини. Малювання через MouseListener/MouseMotionListener. Кнопка «Очистити».

Варіант 14. Таблиця оцінок: JTable з DefaultTableModel та стовпцями «Студент», «Предмет 1»...«Предмет 5». Кнопки «Додати рядок», «Видалити рядок». У рядку стану (нижня JLabel) — середній бал виділеного студента.

Варіант 15. Менеджер контактів: вікно з трьома панелями (JSplitPane): ліво — JList контактів, центр — деталі у JPanel з GridLayout, право — JTextArea нотаток. Кнопки «Додати», «Видалити», «Зберегти зміни».

Контрольні запитання

1. Що таке EDT (Event Dispatch Thread) у Swing? Чому небезпечно оновлювати GUI поза EDT?
 2. Що таке SwingUtilities.invokeLater та SwingWorker? Коли використовувати кожен?
 3. Що таке менеджер розміщення (Layout Manager)? Чому не рекомендується використовувати AbsoluteLayout (null layout)?
 4. Яка різниця між JFrame.EXIT_ON_CLOSE та WindowAdapter.windowClosing?
 5. Що таке Look and Feel (L&F)? Які стандартні L&F постачаються з JDK?
 6. Що таке JScrollPane і навіщо його застосовувати з JTextArea або JList?
 7. Яка різниця між ActionListener та MouseListener? Наведіть приклад, де кожен є доречним.
 8. Що таке DefaultTableModel? Як динамічно додавати рядки до JTable?
 9. Що таке DocumentListener? Для якої задачі він використовується?
 10. Що таке ButtonGroup? Навіщо вона потрібна для JRadioButton?
 11. Як відкрити файл за допомогою JFileChooser? Як встановити фільтр для типів файлів?
 12. Що таке JOptionPane.showInputDialog? Як обробити натискання кнопки «Скасувати»?
 13. Що таке ListCellRenderer? Як змінити відображення окремих елементів у JList?
 14. Як отримати текст з JTextField і встановити його програмно?
 15. Що таке javax.swing.Timer? Чим він відрізняється від java.util.Timer для задач GUI?
-

Перелік посилань

1. Horstmann, C. S. *Core Java, Volume II — Advanced Features* (12th ed.). Hoboken : Pearson, 2022. 832 p.
2. Schildt, H. *Java: The Complete Reference* (12th ed.). New York : McGraw-Hill, 2021. 1248 p.
3. Oracle Corporation. *Creating a GUI With Swing — Java Tutorials* [Електронний ресурс]. URL: <https://docs.oracle.com/javase/tutorial/uiswing/> (дата звернення: 2025-09-01).
4. Oracle Corporation. *javax.swing Package API* [Електронний ресурс]. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.desktop/javax/swing/summary.html> (дата звернення: 2025-09-01).

5. Oracle Corporation. *Concurrency in Swing* [Електронний ресурс]. URL: <https://docs.oracle.com/javase/tutorial/uiswing/concurrency/> (дата звернення: 2025-09-01).
6. Loy, M., Eckstein, R., Wood, D., Elliott, J., Cole, B. *Java Swing* (2nd ed.). Sebastopol : O'Reilly Media, 2002. 1278 p.
7. Bloch, J. *Effective Java* (3rd ed.). Boston : Addison-Wesley, 2018. 412 p.

Лабораторна робота №8

Використання Java для створення мережесих програм

Тема

Використання Java для створення мережесих програм: TCP/IP-сокети, HTTP-клієнти та основи мережесих програмування.

Мета

Набути практичних навичок розробки мережесих застосунків мовою Java шляхом реалізації TCP-клієнта та серверу, а також HTTP-клієнта з використанням стандартного API `java.net.http`.

Теоретичні відомості

Основи мережесих моделі Мережесих комунікація Java базується на стеку протоколів **TCP/IP**: - **IP** (Internet Protocol) — адресація хостів (IPv4: 192.168.1.1; IPv6: ::1). - **TCP** (Transmission Control Protocol) — надійна, орієнтована на з'єднання передача; гарантує доставку та порядок пакетів. - **UDP** (User Datagram Protocol) — ненадійна, без з'єднання; швидша, але без гарантій. - **Порт** — 16-бітний ідентифікатор (0-65 535) для розрізнення сервісів на одному хості. Зарезервовані: 0-1023; загальновідомі: HTTP 80, HTTPS 443, FTP 21.

TCP-сервер (ServerSocket)

```
import java.net.*;
import java.io.*;

public class EchoServer {
    public static void main(String[] args) throws IOException {
        int port = 9000;
```

```

try (ServerSocket serverSocket = new ServerSocket(port)) {
    System.out.println("Сервер слухає порт " + port);
    while (true) {
        // Очікування підключення клієнта (блокуюча операція)
        Socket client = serverSocket.accept();
        System.out.println("Підключено: " + client.getInetAddress());

        // Обробка у потоці, щоб не блокувати прийняття нових з'єднань
        new Thread(() -> handleClient(client)).start();
    }
}

private static void handleClient(Socket socket) {
    try (socket;
        BufferedReader in = new BufferedReader(
            new InputStreamReader(socket.getInputStream(), StandardCharsets.UTF_8));
        PrintWriter out = new PrintWriter(
            new OutputStreamWriter(socket.getOutputStream(), StandardCharsets.UTF_8))) {

        String line;
        while ((line = in.readLine()) != null) {
            System.out.println("Отримано: " + line);
            out.println("Ехо: " + line); // Відправити назад
        }
    } catch (IOException e) {
        System.err.println("Помилка клієнта: " + e.getMessage());
    }
}

```

TCP-клієнт (Socket)

```

public class EchoClient {
    public static void main(String[] args) throws IOException {
        try (Socket socket = new Socket("localhost", 9000);
            PrintWriter out = new PrintWriter(
                new OutputStreamWriter(socket.getOutputStream(), StandardCharsets.UTF_8));
            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream(), StandardCharsets.UTF_8));
            BufferedReader console = new BufferedReader(new InputStreamReader(System.in))) {

            String input;
            System.out.println("Підключено до сервера. Введіть повідомлення (введіть 'exit' для виходу)");
            while (!(input = console.readLine()).equals("exit")) {
                out.println(input);
                System.out.println("Сервер відповів: " + in.readLine());
            }
        }
    }
}

```



```

    }
}
}

```

HTTP-клієнт (java.net.http, Java 11+)

```

import java.net.http.*;
import java.net.URI;
import java.time.Duration;

```

```

HttpClient client = HttpClient.newBuilder()
    .connectTimeout(Duration.ofSeconds(10))
    .build();

```

// GET-запит

```

HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://httpbin.org/get"))
    .header("Accept", "application/json")
    .GET()
    .build();

```

```

HttpResponse<String> response = client.send(request, HttpResponse.BodyHandler
System.out.println("Статус: " + response.statusCode());
System.out.println("Тіло: " + response.body());

```

// POST-запит

```

String json = "{\"sensor\": \"T-01\", \"value\": 23.5}";
HttpRequest postRequest = HttpRequest.newBuilder()
    .uri(URI.create("https://httpbin.org/post"))
    .header("Content-Type", "application/json")
    .POST(HttpRequest.BodyPublishers.ofString(json))
    .build();

```

```

HttpResponse<String> postResponse = client.send(postRequest, HttpResponse.Bod

```

Асинхронний HTTP (sendAsync)

```

client.sendAsync(request, HttpResponse.BodyHandlers.ofString())
    .thenApply(HttpResponse::body)
    .thenAccept(System.out::println)
    .join(); // дочекатися завершення

```

UDP (датаграми)

// Сервер UDP

```

DatagramSocket serverSocket = new DatagramSocket(9001);
byte[] buffer = new byte[1024];
DatagramPacket packet = new DatagramPacket(buffer, buffer.length);

```

```
serverSocket.receive(packet); // блокування до отримання  
String msg = new String(packet.getData(), 0, packet.getLength(), StandardCharsets.UTF_8);
```

Приклад з репозиторію java-iot Клас `NbuApiClient.java` визначає інтерфейс для HTTP-клієнта НБУ. Клас `RealNbuApiClient.java` реалізує запит до реального REST API. Клас `MockNbuApiClient.java` — тестова заглушка. Рекомендується вивчити патерн розділення інтерфейсу та реалізації для тестованого HTTP-клієнта.

Хід виконання роботи

1. Відкрити та проаналізувати `NbuApiClient.java`, `RealNbuApiClient.java` та `MockNbuApiClient.java` (посилання вище). Визначити: який HTTP-метод використовується, як обробляється JSON-відповідь, як тестується код без реальної мережі.
 2. Реалізувати простий TCP-луна-сервер (`EchoServer`): приймає підключення, читає рядки та повертає їх назад із префіксом.
 3. Реалізувати TCP-клієнт (`EchoClient`): підключається до серверу, відправляє введені з консолі рядки та виводить відповідь.
 4. Запустити сервер та клієнт одночасно (два термінальних вікна або `Thread`). Переконалися у двосторонньому обміні.
 5. Реалізувати завдання відповідно до свого варіанту.
 6. Реалізувати HTTP GET-запит за допомогою `java.net.http.HttpClient`. Якщо варіант передбачає публічне API — перевірити реальну відповідь; якщо ні — використати `https://httpbin.org` або `https://jsonplaceholder.typicode.com`.
 7. Обробити відповідь: розібрати JSON-рядок власноруч (методами `String.indexOf`, `substring`) або використати бібліотеку `Gson/Jackson` (якщо дозволяє конфігурація проекту).
 8. Реалізувати обробку мережевих виключень: `ConnectException`, `SocketTimeoutException`, `HttpTimeoutException`. Вивести інформативне повідомлення.
 9. Написати тест із `MockNbuApiClient`-підходом: `mock`-клас, що повертає заготовлену JSON-відповідь, дозволяє тестувати бізнес-логіку без мережі.
 10. Виміряти час відповіді 5 HTTP-запитів (послідовних та асинхронних через `sendAsync`). Порівняти результати.
 11. Оформити звіт: вихідний код сервера та клієнта, знімки виводу при роботі, обговорення мережевих виключень.
-

Індивідуальні завдання

Розробити мережевий застосунок (3–4 класи). Обсяг: до 250 рядків коду. Обов'язково: обробка виключень, підтримка кирилиці (UTF-8).

Варіант 1. Реалізувати «Чат» у CLI: TCP-сервер, що підтримує до 5 одночасних клієнтів. Кожне повідомлення від клієнта транслюється всім іншим із зазначенням імені відправника. Ім'я клієнт надсилає першим рядком при підключенні.

Варіант 2. Реалізувати HTTP-клієнт для API НБУ (за зразком [RealNbuApiClient.java](#)): GET-запит до <https://bank.gov.ua/NBUStatService/v1/statdata> розбір JSON-відповіді (рядками), вивід курсів USD, EUR, GBP.

Варіант 3. Реалізувати HTTP-клієнт для відкритого API <https://jsonplaceholder.typicode.com/> отримати список постів (GET), вивести перші 5. Реалізувати POST-запит для створення нового поста. Вивести статус відповіді та тіло.

Варіант 4. Реалізувати TCP-сервер «Вікторина»: при підключенні клієнт отримує питання, надсилає відповідь, сервер перевіряє та відповідає «Правильно!» або «Неправильно. Правильна відповідь: ...». Після 5 питань — підсумковий рахунок.

Варіант 5. Реалізувати клієнт-серверний «Словник»: TCP-сервер зберігає Map<String, String> (слово → переклад). Клієнт надсилає слово, сервер повертає переклад або «Не знайдено». Підтримати команди ADD слово переклад та DEL слово.

Варіант 6. Реалізувати HTTP-клієнт для відкритого прогнозу погоди: API <https://wttr.in/<місто>?format=j1> повертає JSON. Розібрати рядками та вивести температуру, опис погоди, вологість для Тернополя або введеного міста.

Варіант 7. Реалізувати TCP «Файловий сервер»: клієнт надсилає ім'я файлу → сервер читає файл і надсилає вміст байт за байтом через InputStream. Клієнт зберігає отримані байти у файл. Передбачити обробку відсутнього файлу.

Варіант 8. Реалізувати HTTP-клієнт для API GitHub: GET-запит до <https://api.github.com/repos/<owner>/<repo>> (публічний репозиторій). Вивести: назву, опис, кількість зірок, мову. Параметри репо — аргументи main.

Варіант 9. Реалізувати UDP-застосунок «Датчик — Реєстратор»: клієнт надсилає UDP-датаграму з показанням (рядок "Т:23.5 Н:65"), сервер отримує, зберігає у List та виводить на консоль. Після 10 датаграм — вивести статистику.

Варіант 10. Реалізувати TCP-сервер «Калькулятор»: клієнт надсилає вираз у рядку ("3 + 4", "10 / 2"), сервер обчислює та повертає результат. Підтримати +, -, *, /. Обробити ділення на нуль.

Варіант 11. Реалізувати HTTP-клієнт для перевірки IP-геолокації: GET-запит до <https://ipinfo.io/{ip}/json> (публічне API). Розібрати відповідь та вивести: місто, регіон, країну, провайдера. Підтримати введення IP як аргумент main.

Варіант 12. Реалізувати «Менеджер задач» на TCP: сервер зберігає список Task(id, title, done). Клієнт підтримує команди: LIST, ADD

назва, DONE id, DELETE id. Відповіді сервера — рядки у форматі CSV.

Варіант 13. Реалізувати HTTP-клієнт для `https://jsonplaceholder.typicode.com/u` завантажити список користувачів (GET), вивести таблицю (id, ім'я, email, місто). Виконати PUT-запит для оновлення першого користувача та перевірити відповідь.

Варіант 14. Реалізувати TCP «Нотатки» з трьома паралельними клієнтами: `Map<String, String>` (ключ → нотатка). Команди: PUT key value, GET key, LIST. Захистити Map від конкурентного доступу через `ConcurrentHashMap`.

Варіант 15. Реалізувати HTTP-клієнт для REST API `https://restcountries.com/v3`. запит за назвою країни (аргумент main). Вивести: офіційна назва, столиця, населення, регіон. Обробити 404 (країна не знайдена).

Контрольні запитання

1. Яка різниця між TCP та UDP? Наведіть приклад задачі, де доцільний кожен протокол.
 2. Що таке сокет? Яку пару параметрів однозначно ідентифікує TCP-з'єднання?
 3. Що таке `ServerSocket.accept()`? Чому цей виклик є блокуючим?
 4. Навіщо обробляти кожне з'єднання у окремому потоці? Які ризики пов'язані з цим підходом?
 5. Що таке `SocketTimeoutException`? Як встановити тайм-аут очікування підключення та читання?
 6. Яке призначення `HttpClient` у `java.net.http`? Чим він кращий за `URLConnection`?
 7. Яка різниця між синхронним `send` та асинхронним `sendAsync` у `HttpClient`?
 8. Що таке `BodyPublisher` та `BodyHandler`? Для чого вони потрібні?
 9. Що таке REST API? Яку роль відіграють HTTP-методи GET, POST, PUT, DELETE?
 10. Що таке JSON? Наведіть приклад структури JSON-об'єкта та JSON-масиву.
 11. Що таке `ConnectException` та коли вона виникає?
 12. Яка різниця між `InetAddress` та `URI`? Для яких задач вони використовуються?
 13. Що таке `DatagramSocket` та `DatagramPacket`? Як надіслати UDP-повідомлення?
 14. Що таке `localhost` та `loopback`-адреса? Як пов'язані `127.0.0.1` та `::1`?
 15. Що таке HTTP-заголовки (headers)? Яку роль відіграють `Content-Type` та `Accept`?
-

Перелік посилань

1. Horstmann, C. S. *Core Java, Volume II — Advanced Features* (12th ed.). Hoboken : Pearson, 2022. 832 p.
2. Schildt, H. *Java: The Complete Reference* (12th ed.). New York : McGraw-Hill, 2021. 1248 p.
3. Oracle Corporation. *Custom Networking — Java Tutorials* [Електронний ресурс]. URL: <https://docs.oracle.com/javase/tutorial/networking/> (дата звернення: 2025-09-01).
4. Oracle Corporation. *java.net.http — HTTP Client API* [Електронний ресурс]. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.net.http/summary.html> (дата звернення: 2025-09-01).
5. Kurose, J. F., Ross, K. W. *Computer Networking: A Top-Down Approach* (8th ed.). Hoboken : Pearson, 2021. 856 p.
6. Richardson, L., Ruby, S. *RESTful Web Services*. Sebastopol : O'Reilly Media, 2007. 448 p.
7. Tanenbaum, A. S., Wetherall, D. J. *Computer Networks* (5th ed.). Upper Saddle River : Pearson, 2011. 960 p.

Лабораторна робота №9

Створення програм з кількома підпроцесами

Тема

Створення програм з кількома підпроцесами (багатопотокове програмування): потоки, синхронізація та паралельне виконання у Java.

Мета

Набути практичних навичок розробки багатопотокових застосунків мовою Java шляхом реалізації потоків виконання, механізмів синхронізації та паралельних обчислень з використанням `ExecutorService`, `synchronized`, `volatile` та `CompletableFuture`.

Теоретичні відомості

Потоки виконання (Threads) Потік (*thread*) — найменша одиниця виконання у межах процесу. У Java кожен потік є екземпляром класу `java.lang.Thread`. Два способи створення потоку:

```
// Спосіб 1: успадкування Thread  
class CounterThread extends Thread {
```

```

private final int id;
public CounterThread(int id) { this.id = id; }

@Override
public void run() {
    for (int i = 0; i < 5; i++) {
        System.out.println("Потік " + id + ": " + i);
    }
}
}

```

// Спосіб 2: реалізація Runnable (переважається)

```

Thread t = new Thread(() -> System.out.println("Лямбда-потік!"));
t.start();

```

// Запуск

```

new CounterThread(1).start();

```

Важливо: `thread.run()` виконує метод у поточному потоці. `thread.start()` створює новий системний потік і запускає в ньому `run()`.

Стани потоку Потік знаходиться в одному з станів: NEW → RUNNABLE → (BLOCKED | WAITING | TIMED_WAITING) → TERMINATED.

ExecutorService — пул потоків Ручне керування потоками ненадійне. Рекомендується використовувати пул:

```

import java.util.concurrent.*;

```

// Фіксований пул з 4 потоками

```

ExecutorService executor = Executors.newFixedThreadPool(4);

```

```

for (int i = 0; i < 10; i++) {
    final int taskId = i;
    executor.submit(() -> System.out.println("Задача " + taskId));
}

```

```

executor.shutdown();

```

// зупинка після завершення задач

```

executor.awaitTermination(30, TimeUnit.SECONDS);

```

Callable та Future

```

Callable<Double> task = () -> {
    Thread.sleep(1000);
    return Math.PI;
};

```

```

Future<Double> future = executor.submit(task);
System.out.println("Обчислення... ");

```

```
double result = future.get(); // блокує до отримання результату  
System.out.println("PI ~ " + result);
```

CompletableFuture (Java 8+)

```
CompletableFuture.supplyAsync(() -> fetchTemperature())  
    .thenApply(temp -> "Температура: " + temp + "°C")  
    .thenAccept(System.out::println)  
    .exceptionally(ex -> { System.err.println(ex.getMessage()); return null; })  
    .join();
```

Синхронізація: synchronized При одночасному доступі кількох потоків до спільних даних виникають **гонки даних** (*data races*). Для захисту використовується `synchronized`:

```
public class SafeCounter {  
    private int count = 0;  
  
    // Синхронізований метод – захищає весь метод  
    public synchronized void increment() { count++; }  
  
    // Синхронізований блок – мінімальна критична секція  
    public void decrement() {  
        synchronized (this) { count--; }  
    }  
  
    public synchronized int get() { return count; }  
}
```

volatile `volatile` гарантує **видимість** змін між потоками (але не атомарність складних операцій):

```
volatile boolean running = true;  
  
// Потік, що перевіряє прапорець зупинки  
while (running) { /* виконання */ }  
  
// Інший потік  
running = false; // з volatile – гарантовано побачить перший потік
```

Атомарні класи

```
import java.util.concurrent.atomic.*;  
  
AtomicInteger counter = new AtomicInteger(0);  
counter.incrementAndGet(); // атомарний інкремент  
counter.compareAndSet(5, 10); // порівняти і замінити
```

Блокування: Lock та ReentrantLock

```
import java.util.concurrent.locks.*;

Lock lock = new ReentrantLock();
lock.lock();
try {
    // критична секція
} finally {
    lock.unlock(); // завжди у finally
}
```

Віртуальні потоки (Virtual Threads, Java 21+)

```
// Надлегкі потоки керовані JVM (не ОС)
Thread vt = Thread.ofVirtual().start(() -> System.out.println("Віртуальний потік"));
ExecutorService vtExecutor = Executors.newVirtualThreadPerTaskExecutor();
```

Віртуальні потоки дозволяють мати мільйони одночасних «потоків» без перевантаження ОС.

Приклад з репозиторію java-iot Клас [ReactionEngine.java](#) координує роботу ігрових раундів через час та стимули у кількох потоках. Клас [GracefulShutdown.java](#) демонструє коректне завершення роботи через shutdown hook та зупинку потоків. Рекомендується проаналізувати ці класи.

Хід виконання роботи

1. Відкрити та проаналізувати [ReactionEngine.java](#) та [GracefulShutdown.java](#) (посилання вище). Визначити: які механізми синхронізації використано, як керується зупинка потоків.
2. Реалізувати два потоки, що паралельно збільшують спільний лічильник (без синхронізації). Запустити 1000 разів та зафіксувати ненульову ймовірність гонки.
3. виправити гонку через `synchronized`, потім через `AtomicInteger`. Порівняти коректність та продуктивність.
4. Реалізувати пул потоків для паралельного виконання задач відповідно до варіанту.
5. Реалізувати `Callable`-задачу, що повертає результат через `Future`. Виміряти час виконання послідовно та паралельно.
6. Продемонструвати `volatile`: запустити потік у циклі з прапорцем зупинки, переконатися, що без `volatile` потік може не зупинитися.
7. Реалізувати виробника-споживача (`Producer-Consumer`) через `BlockingQueue<String>`: один потік-виробник, один потік-споживач.
8. Реалізувати завдання відповідно до варіанту з використанням `CompletableFuture`.

9. Продемонструвати `Thread.sleep`, `Thread.join`, `interrupt`. Обробити `InterruptedException`.
 10. Написати 3 тести, що перевіряють потокобезпечність (використати `CountDownLatch` або `CyclicBarrier` для синхронізації початку).
 11. Оформити звіт: вихідний код, діаграма взаємодії потоків, результати порівняльного вимірювання часу.
-

Індивідуальні завдання

Розробити багатопотокову програму (2–3 класи). Обсяг: до 150 рядків коду. Обов'язково: коректна синхронізація, відсутність `deadlock`, коректне завершення потоків.

Варіант 1. Реалізувати паралельне завантаження даних: 5 `Callable<String>` симулюють завантаження з різних джерел (через `Thread.sleep` з різними затримками). Зібрати результати через `List<Future<String>>`, вивести у порядку завершення через `CompletionService`.

Варіант 2. Реалізувати «Виробник-споживач» для IoT-даних (натхненний `ReactionEngine.java`): один потік-«датчик» кожні 500 мс додає показання у `BlockingQueue<Double>` (ємність 10), потік-«реєстратор» забирає та записує у файл. Завершення через `volatile boolean running`.

Варіант 3. Реалізувати паралельне сортування: масив з 10 000 елементів розбити на N частин (N = кількість ядер), відсортувати кожен частину у окремому `CompletableFuture`, потім злити. Порівняти час із `Arrays.sort`.

Варіант 4. Реалізувати «Банк» з потокобезпечними рахунками: клас `BankAccount` з `ReentrantLock`. N потоків виконують випадкові перекази між рахунками. Перевірити інваріант: сума всіх рахунків до = сума після всіх переказів.

Варіант 5. Реалізувати паралельний пошук у файлах: N потоків шукають рядок у N різних файлах (або частинах одного файлу). Використати `ExecutorService` та `Future<Long>` (кількість збігів). Вивести загальний результат.

Варіант 6. Реалізувати «Ліфт»: `ElevatorSimulator` з `BlockingQueue<Integer>` (запити поверхів). Один потік-«ліфт» обробляє запити; кілька потоків-«пасажирів» додають запити. Вивести лог переміщень.

Варіант 7. Реалізувати `PeriodicTaskRunner` (натхненний `GracefulShutdown.java`): `ScheduledExecutorService` виконує задачу кожні N секунд. Коректне завершення через `shutdown hook` (`Runtime.getRuntime().addShutdownHook`). Перевірити: при `Ctrl+C` — задача завершується чисто.

Варіант 8. Реалізувати `ParallelWordCount`: `Files.lines(path)` розбити на частини; кожна частина обробляється у окремому потоці через `CompletableFuture.supplyAsync`; результати (`Map<String, Long>`) об'єднати методом `thenCombine`.

Варіант 9. Реалізувати «Семафорна парковка»: Semaphore на 3 місця; 10 потоків-«автомобілів» намагаються «припаркуватися» (acquire) на випадковий час. Вивести лог: хто зайшов, хто чекає, хто вийшов.

Варіант 10. Реалізувати потокобезпечний кеш: Cache<K, V> на ConcurrentHashMap з TTL (час до застарівання). ScheduledExecutorService кожен хвилину видаляє прострочені записи. Методи: put, get, size.

Варіант 11. Реалізувати RaceGame: N «гравців» (потоки) стартують одночасно через CountdownLatch. Кожен «гравець» — цикл з Thread.sleep(random). Перший, хто завершить 10 ітерацій, — переможець. Вивести час кожного гравця.

Варіант 12. Реалізувати паралельну обробку зображення (симуляція): 2D-масив int[][] пікселів. ForkJoinPool рекурсивно ділить масив на рядки, кожен рядок обробляє (pixel * 2 — підвищення яскравості). Порівняти час з послідовною обробкою.

Варіант 13. Реалізувати «Читачі-письменники» (ReadWriteLock): клас SharedDocument зі спільним рядком, захищеним ReentrantReadWriteLock. 5 потоків-«читачів» та 2 потоки-«письменники». Вивести лог конкурентного доступу.

Варіант 14. Реалізувати TaskPipeline з трьома етапами: Stage1 читає з джерела → передає у BlockingQueue; Stage2 трансформує → передає у BlockingQueue; Stage3 записує у файл. Кожен етап — окремий потік.

Варіант 15. Реалізувати «Аукціон»: клас Auction з volatile double currentBid. N потоків-«учасників» роблять ставки через compareAndExchange (VarHandle) або AtomicReference. Переможець — найвища ставка після завершення аукціону. Перевірити відсутність гонки.

Контрольні запитання

1. Що таке потік виконання (thread)? Яка різниця між потоком та процесом?
2. Чим Thread.start() відрізняється від Thread.run()? Що відбудеться при виклику run() без start()?
3. Що таке гонка даних (data race)? Наведіть приклад коду, що демонструє гонку.
4. Що таке synchronized? Яка різниця між синхронізованим методом та синхронізованим блоком?
5. Що таке volatile? Яку гарантію воно дає і яку — ні?
6. Що таке AtomicInteger? Як compareAndSet дозволяє реалізувати атомарні операції без synchronized?
7. Що таке ExecutorService? Яка різниця між Executors.newFixedThreadPool та newCachedThreadPool?
8. Що таке Future.get()? Що відбудеться, якщо задача кинула виключення?

9. Що таке `CompletableFuture`? Чим він відрізняється від `Future` з точки зору функціональної композиції?
 10. Що таке `deadlock`? Опишіть умови його виникнення та способи запобігання.
 11. Що таке `BlockingQueue`? Для якого паттерну він найчастіше використовується?
 12. Що таке `Semaphore`? Наведіть приклад задачі, де він є зручним.
 13. Що таке `CountDownLatch`? Чим він відрізняється від `CyclicBarrier`?
 14. Що таке віртуальні потоки (`Virtual Threads`)? У чому їхня перевага над платформними потоками?
 15. Що таке `ThreadLocal`? Для яких задач він використовується?
-

Перелік посилань

1. Goetz, B., Peierls, T., Bloch, J., Bowbeer, J., Holmes, D., Lea, D. *Java Concurrency in Practice*. Boston : Addison-Wesley, 2006. 403 p.
2. Horstmann, C. S. *Core Java, Volume II — Advanced Features* (12th ed.). Hoboken : Pearson, 2022. 832 p.
3. Bloch, J. *Effective Java* (3rd ed.). Boston : Addison-Wesley, 2018. 412 p.
4. Oracle Corporation. *Concurrency — Java Tutorials* [Електронний ресурс]. URL: <https://docs.oracle.com/javase/tutorial/essential/concurrency/> (дата звернення: 2025-09-01).
5. Oracle Corporation. *Virtual Threads — JEP 444* [Електронний ресурс]. URL: <https://openjdk.org/jeps/444> (дата звернення: 2025-09-01).
6. Oracle Corporation. *java.util.concurrent Package API* [Електронний ресурс]. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/package-summary.html> (дата звернення: 2025-09-01).
7. Schildt, H. *Java: The Complete Reference* (12th ed.). New York : McGraw-Hill, 2021. 1248 p.

Лабораторна робота №10

Робота з базами даних

Тема

Робота з реляційними базами даних у Java засобами JDBC: підключення, виконання SQL-запитів, транзакції та патерн репозиторію.

Мета

Набути практичних навичок взаємодії з реляційними базами даних у Java шляхом реалізації програм, що використовують JDBC API для виконання SQL-запитів SELECT, INSERT, UPDATE, DELETE, керування транзакціями та патерн репозиторію.

Теоретичні відомості

Архітектура JDBC **JDBC (Java Database Connectivity)** — стандартний API для взаємодії Java-застосунків з реляційними базами даних. Архітектура:

Застосунок Java
↓ (JDBC API)
JDBC Driver Manager
↓ (JDBC Driver)
База даних (SQLite / PostgreSQL / MySQL / H2 тощо)
Ключові класи в пакеті java.sql:

Клас/Інтерфейс	Призначення
DriverManager	Керування драйверами, отримання з'єднань
Connection	З'єднання з БД; управління транзакціями
Statement	Виконання простих SQL-запитів
PreparedStatement	Параметризовані запити (захист від SQL-ін'єкції)
CallableStatement	Виклик збережених процедур
ResultSet	Набір рядків результату запиту

SQLite як навчальна база даних **SQLite** — легковагова вбудована СУБД, що зберігає базу у єдиному файлі. Не потребує окремого серверного процесу. Ідеально для навчального середовища. JDBC-драйвер: org.xerial:sqlite-jdbc.

```
<!-- Maven -->
<dependency>
  <groupId>org.xerial</groupId>
  <artifactId>sqlite-jdbc</artifactId>
  <version>3.45.3.0</version>
</dependency>
```

Підключення та базові операції

```
import java.sql.*;

public class DatabaseExample {

    private static final String URL = "jdbc:sqlite:mydata.db";

    public static void main(String[] args) throws SQLException {
        // Підключення – автоматично закривається у try-with-resources
        try (Connection conn = DriverManager.getConnection(URL)) {

            // Створення таблиці
            try (Statement stmt = conn.createStatement()) {
                stmt.execute("""
                    CREATE TABLE IF NOT EXISTS sensors (
                        id      INTEGER PRIMARY KEY AUTOINCREMENT,
                        name    TEXT      NOT NULL,
                        value   REAL,
                        ts       INTEGER NOT NULL
                    )
                """);
            }

            // Вставка (НИКОЛИ не конкатенувати значення у SQL – SQL-ін'єкція)
            String sql = "INSERT INTO sensors (name, value, ts) VALUES (?, ?, ?)";
            try (PreparedStatement ps = conn.prepareStatement(sql)) {
                ps.setString(1, "DHT11");
                ps.setDouble(2, 23.5);
                ps.setLong(3, System.currentTimeMillis());
                ps.executeUpdate();
            }

            // Запит
            try (PreparedStatement ps = conn.prepareStatement(
                "SELECT id, name, value, ts FROM sensors WHERE name = ?")) {
                ps.setString(1, "DHT11");
                try (ResultSet rs = ps.executeQuery()) {
                    while (rs.next()) {
                        System.out.printf("ID=%d  name=%s  value=%.1f%n",
                            rs.getInt("id"),
                            rs.getString("name"),
                            rs.getDouble("value"));
                    }
                }
            }
        }
    }
}
```

Транзакції За замовчуванням JDBC увімкнено режим `autoCommit=true` — кожен оператор фіксується одразу. Для атомарного виконання кількох операцій:

```
conn.setAutoCommit(false);
try {
    debit(conn, fromAccount, amount);
    credit(conn, toAccount, amount);
    conn.commit();           // зафіксувати обидві операції разом
} catch (SQLException e) {
    conn.rollback();         // відкотити обидві при помилці
    throw e;
} finally {
    conn.setAutoCommit(true);
}
```

Патерн репозиторію (Repository Pattern) Відокремлення логіки доступу до бази від бізнес-логіки:

// Модель даних

```
public record SensorReading(long id, String name, double value, long timestamp)
```

// Інтерфейс репозиторію

```
public interface SensorReadingRepository {
    void save(SensorReading reading) throws SQLException;
    List<SensorReading> findAll() throws SQLException;
    List<SensorReading> findByName(String name) throws SQLException;
    void deleteById(long id) throws SQLException;
}
```

// Реалізація

```
public class SqliteSensorReadingRepository implements SensorReadingRepository {
    private final Connection conn;

    public SqliteSensorReadingRepository(Connection conn) { this.conn = conn; }

    @Override
    public void save(SensorReading r) throws SQLException {
        try (PreparedStatement ps = conn.prepareStatement(
            "INSERT INTO sensors(name, value, ts) VALUES(?,?,?)")) {
            ps.setString(1, r.name());
            ps.setDouble(2, r.value());
            ps.setLong(3, r.timestamp());
            ps.executeUpdate();
        }
    }
    // ... інші методи
}
```

Запобігання SQL-ін'єкціям Ніколи не будувати SQL-рядки конкатенацією з користувацьким введенням:

// НЕБЕЗПЕЧНО!

```
String badSql = "SELECT * FROM users WHERE name = '" + userInput + "'";
```

// ПРАВИЛЬНО – PreparedStatement

```
PreparedStatement ps = conn.prepareStatement("SELECT * FROM users WHERE name  
ps.setString(1, userInput);
```

Приклад з репозиторію java-iot Клас [DatabaseManager.java](#) ініціалізує SQLite-базу та виконує DDL (CREATE TABLE). Клас [DeviceEventRepository.java](#) реалізує CRUD-операції для подій пристроїв. Клас [ProfileRepository.java](#) зберігає профілі налаштувань. Рекомендується вивчити ці класи як зразок промислового використання JDBC.

Хід виконання роботи

1. Відкрити та проаналізувати [DatabaseManager.java](#), [DeviceEventRepository.java](#) та [ProfileRepository.java](#) (посилання вище). Визначити: як встановлюється з'єднання, які SQL-запити виконуються, як закриваються ресурси.
 2. Підключити JDBC-драйвер SQLite до проєкту (Gradle або Maven-залежність). Переконавшись у можливості підключення командою `DriverManager.getConnection("jdbc:sqlite:test.db")`.
 3. Створити схему бази даних (таблиці) відповідно до варіанту. Використати `CREATE TABLE IF NOT EXISTS` у методі ініціалізації.
 4. Реалізувати клас репозиторію з методами `save`, `findAll`, `findById`, `update`, `deleteById`. Усі методи — через `PreparedStatement`.
 5. Реалізувати `main`-метод, що демонструє повний CRUD-цикл: вставка 5+ записів, читання всіх, оновлення одного, видалення одного, повторне читання.
 6. Реалізувати транзакцію для операції, що вимагає атомарності (залежно від варіанту): вимкнути `autoCommit`, виконати кілька операцій, `commit/rollback`.
 7. Реалізувати запит з `WHERE`, `ORDER BY` та `LIMIT`. Вивести результати у форматованій таблиці.
 8. Реалізувати пошук за декількома параметрами з `AND/OR`.
 9. Написати 5 JUnit 5 тестів з `@BeforeEach` (створення in-memory SQLite `jdbc:sqlite::memory:`) та `@AfterEach` (закриття з'єднання).
 10. Оформити звіт: схема бази даних (у вигляді SQL `CREATE TABLE` або таблиць), вихідний код репозиторію, результати CRUD-демонстрації, результати тестів.
-

Індивідуальні завдання

Розробити програму (3–4 класи: схема БД, репозиторій, модель, демонстрація). Обсяг: до 150 рядків коду. СУБД: SQLite (in-file або in-memory для тестів).

Варіант 1. База даних студентів: таблиці `students(id, name, group_name, year)` та `grades(id, student_id, subject, grade)`. Репозиторій: `addStudent`, `addGrade`, `getAverageGrade(int studentId)`, `getTopStudents(int n)`. Тести: порожня БД, топ-список.

Варіант 2. База даних бібліотеки: таблиці `books(id, title, author, year, available)` та `loans(id, book_id, reader_name, loan_date, return_date)`. Репозиторій: `borrowBook` (транзакція: позначити `available=false` + додати запис у `loans`), `returnBook`.

Варіант 3. База даних IoT-датчиків (натхненний [DeviceEventRepository.java](#)): таблиця `events(id, device_id, event_type, timestamp, payload TEXT)`. Методи: `logEvent`, `getLastN(int n)`, `getByDevice(String id)`, `cleanup(long olderThanMs)`.

Варіант 4. Інтернет-магазин: таблиці `products(id, name, price, stock)` та `orders(id, product_id, quantity, total, created_at)`. Репозиторій: `placeOrder` (транзакція: перевірити наявність → зменшити `stock` → додати замовлення). Тест на недостатній залишок.

Варіант 5. Контакт-книга: таблиця `contacts(id, name, email, phone, notes)`. Репозиторій: `add`, `findByName(String)` (LIKE-пошук), `findByPhone`, `update`, `delete`. Всі методи — `PreparedStatement`. Тести: пошук з кирилицею.

Варіант 6. Менеджер задач: таблиця `tasks(id, title, description, status, priority, created_at, due_date)`. Статус: `TODO`, `IN_PROGRESS`, `DONE`. Методи: `create`, `updateStatus`, `getByStatus`, `getOverdue(long now)`. Тест транзакції оновлення.

Варіант 7. Профілі розумного будинку (натхненний [ProfileRepository.java](#)): таблиця `profiles(id, name, settings_json)`. Зберігати налаштування як JSON-рядок. Методи: `saveProfile`, `loadProfile(String name)`, `listProfiles`, `deleteProfile`. Тест завантаження відсутнього профілю.

Варіант 8. Журнал вимірювань: таблиця `measurements(id, sensor_name, value, unit, timestamp)`. Методи: `record`, `getAverage(String sensor, long from, long to)`, `getMin`, `getMax`, `getCount`. Всі агрегати — через SQL-функції, не Java. Тести на правильність агрегатів.

Варіант 9. Банківська система: таблиці `accounts(id, owner, balance)` та `transfers(id, from_id, to_id, amount, ts)`. Метод `transfer(long fromId, long toId, double amount)` — транзакція. Тест: паралельні перекази без втрати балансу (через `SERIALIZABLE` ізоляцію).

Варіант 10. Каталог фільмів: таблиці `movies(id, title, year,`

genre, rating) та reviews(id, movie_id, reviewer, score, comment). Методи: addMovie, addReview, getAverageRating(int movieId), getTopByGenre(String genre, int n). Тести на порядок сортування.

Варіант 11. Реєстр транспортних засобів: таблиця vehicles(id, plate, owner, brand, model, year, insured). Методи: register, findByPlate, findByOwner, updateInsurance(String plate, boolean insured), getExpiredInsurance(). Тести: пошук за точним та частковим номером.

Варіант 12. Система бронювання: таблиці rooms(id, number, type, price_per_night) та reservations(id, room_id, guest_name, check_in, check_out). Метод reserve — транзакція з перевіркою перетину дат. Тести: спроба подвійного бронювання.

Варіант 13. Словник: таблиця words(id, original, translation, language_from, language_to, added_at). Методи: addWord, findTranslation(String word, String langFrom, String langTo), randomWord(String langFrom), deleteWord. Тест з кирилицею.

Варіант 14. Система оцінювання (LMS): таблиці courses(id, name, description), enrollments(id, student_name, course_id, enrolled_at), submissions(id, enrollment_id, score, submitted_at). Метод getFinalScore(String student, String course) — середнє балів. Тест пуского курсу.

Варіант 15. Кімнатні рослини: таблиця plants(id, name, species, last_watered, last_fertilized, location). Методи: add, water(int id), fertilize(int id), getNeedingWater(long maxAgeMs), getNeedingFertilizer. Тест на правильне визначення потребуючих догляду.

Контрольні запитання

1. Що таке JDBC? Яку роль відіграє JDBC-драйвер?
2. Яка різниця між Statement та PreparedStatement? Чому завжди слід надавати перевагу PreparedStatement?
3. Що таке SQL-ін'єкція? Наведіть приклад уразливого коду та його безпечний аналог.
4. Що таке транзакція? Які властивості описує акронім ACID?
5. Що означає autoCommit=true за замовчуванням? Коли потрібно вимикати цей режим?
6. Що таке rollback? Коли він викликається автоматично, а коли — вручну?
7. Що таке ResultSet? Як читати значення стовпців за ім'ям та за індексом?
8. Що відбудеться, якщо Connection, Statement або ResultSet не закрити? Яку конструкцію Java використовувати для гарантованого закриття?
9. Що таке SQL DDL та DML? Наведіть по одному прикладу операторів кожного типу.

10. Що таке JOIN в SQL? Поясніть різницю між INNER JOIN, LEFT JOIN та RIGHT JOIN.
 11. Що таке LIKE в SQL? Як шукати рядки, що містять підрядок?
 12. Що таке ORDER BY та LIMIT? Як вибрати перші 10 записів, відсортованих за датою?
 13. Що таке індекс у базі даних? Коли його варто створювати?
 14. Що таке in-memory база даних SQLite (jdbc:sqlite::memory:)? Чому вона зручна для тестування?
 15. Що таке патерн «Репозиторій» (Repository)? Яку проблему він вирішує в архітектурі застосунку?
-

Перелік посилань

1. Horstmann, C. S. *Core Java, Volume II — Advanced Features* (12th ed.). Hoboken : Pearson, 2022. 832 p.
2. Schildt, H. *Java: The Complete Reference* (12th ed.). New York : McGraw-Hill, 2021. 1248 p.
3. Oracle Corporation. *JDBC Database Access — Java Tutorials* [Електронний ресурс]. URL: <https://docs.oracle.com/javase/tutorial/jdbc/> (дата звернення: 2025-09-01).
4. Oracle Corporation. *java.sql Package API* [Електронний ресурс]. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.sql/java/sql/package-summary.html> (дата звернення: 2025-09-01).
5. SQLite Consortium. *SQLite Documentation* [Електронний ресурс]. URL: <https://www.sqlite.org/docs.html> (дата звернення: 2025-09-01).
6. Bloch, J. *Effective Java* (3rd ed.). Boston : Addison-Wesley, 2018. 412 p.
7. Fowler, M. *Patterns of Enterprise Application Architecture*. Boston : Addison-Wesley, 2002. 560 p.